Numerical Linear Algebra

Rao
Politecnico di Milano

Originally written in: **2024-09-16**

Last updated at: **2025-05-07**

Contents

1 Floating Point Arithmetic

Since computers use a **finite number** of bits to represent a real number, **they can only represent a finite subset of the real numbers**. In general the range of numbers is sufficient large but there are naturally gaps, which might lead to problems.

Hence, it is time to discuss the representation of real numbers in a computer. We are used to representing a number in digits

$$105.67 = 1000 \cdot 0.10567 = +10^3(1 \cdot 10^{-1} + 0 \cdot 10^{-2} + 5 \cdot 10^{-3} + 6 \cdot 10^{-4} + 7 \cdot 10^{-5}) \quad (1.1)$$

Def Floating Point Representation **definition 1.0.1**

A B-adic, normalised floating point number of precision m is either $x = 0$ or

$$x = \pm B^e \sum_{k=-m}^{-1} x_k B^k, \; x_{-1} \neq 0, x_k \in \{0, ..., B-1\}, e \in \mathbb{Z} \quad (1.2)$$

Here, e denote the *exponent* within a given range $e_{\min} \leq e \leq e_{\max}, B \geq 2$ is the *base* and $\sum x_k B_k$ denotes the *mantissa*.

For example, the **IEEE 754** norm states that for a double precision number we have $B = 2$ and $m = 52$. Hence, if one bit is used to store the sign and if 11 bits are reserved for representing the exponent, a double precision number can be stored in 64 bits. In this case, the **IEEE 754** norm defines the following additional values:

- $\pm \inf$ for $\pm\infty$ as the result of dividing positive or negative numbers by zero, for the evaluation of $\log(0)$ and for *overflow* errors, meaning results which would require an exponent outside the range $[e_{\min} \leq e \leq e_{\max}]$.
- *NaN* (not a number) for undefined numbers as they, for example, occur when dividing 0 by 0 or evaluating the logarithm for a negative number.

As a consequence, the distribution of floating point numbers is not uniform. Furthermore, every real number and the result of every operation has to be rounded. There are different rounding strategies, but the default one is rounding x to the nearest machine number $\mathrm{rd}(x)$.

Def Machine Epsilon **definition 1.0.2**

The *machine precision*, eps, is the smallest number such that

$$|x - \mathrm{rd}(x)| \leq \mathrm{eps} \; |x| \quad (1.3)$$

for all $x \in \mathbb{R}$ within the range of the floating point system.

It is easy to determine the machine precision for the default rounding strategy.

Thm Machine Epsilon **theorem 1.0.1**

For a floating point number system with base B and precision m the machine precision is given by $\text{eps} = B^{1-m}$, we have

$$|x - \text{rd}(x)| \leq B^{1-m}|x| \tag{1.4}$$

Thm Error Propagation **theorem 1.0.2**

Let $*$ be one of the operations $+, -, \cdot, \div$ and $\circledast$ be the equivalent floating operation, then for all x, y from the floating point system, there exists an $\varepsilon \in \mathbb{R}$ with $|\varepsilon| \leq \text{eps}$ such that

$$x \circledast y = (x * y)(1 + \varepsilon) \tag{1.5}$$

2 Norms

The essential notions of **size and distance** in a vector space are captured by norms. These are the *yardsticks* with which we measure approximations and convergence throughout numerical linear algebra.

2.1 Vector Norms

A norm is a function $\|\cdot\| : \mathbb{C}^m \to \mathbb{R}$ that assigns a real-valued length to each vector. In order to conform to a reasonable notion of length, a norm must satisfy the following three conditions. For all vectors x, y and for all scalars $\alpha \in \mathbb{C}$:

1. *Nonnegativity*: $\|\boldsymbol{x}\| \geq 0$ and $\|\boldsymbol{x}\| = 0$ if and only if $x = 0$.
2. *Triangle Inequality*: $\|\boldsymbol{x} + \boldsymbol{y}\| \leq \|\boldsymbol{x}\| + \|\boldsymbol{y}\|$.
3. *Homogeneity*: $\|\boldsymbol{\alpha x}\| = |\alpha| \, \|\boldsymbol{x}\|$.

The above conditions allow for different notions of lenght, and at times it is useful to have the flexibility.

$$\begin{aligned} \|\boldsymbol{x}\|_1 &= \sum_{i=1}^{m} |x_i| \\ \|\boldsymbol{x}\|_2 &= \sqrt{\sum_{i=1}^{m} |x_i|^2} \\ \|\boldsymbol{x}\|_\infty &= \max_{\{1<i<m\}} |x_i| \\ \|\boldsymbol{x}\|_p &= \left(\sum_{i=1}^{m} |x_i|^p \right)^{\frac{1}{p}} \; (p \leq 1 < \infty) \end{aligned} \tag{2.1}$$

Aside from the $p-$norms, the most useful norms are the *weighted p norms*, where each of the coordinates of a vector space is given its own weight. In general, given any norm $\|\cdot\|$, the *weighted p norm* is defined as:

$$\|\boldsymbol{x}\|_w = \|\boldsymbol{W x}\| \tag{2.2}$$

Here W is the diagonal matrix with the ith diagonal entry is the weight $w_i \neq 0$. For example. a weighted 2-norm is specified as follows:

$$\|\boldsymbol{x}\|_W = \left(\sum_{i=1}^{m} |w_i x_i|^2 \right)^{\frac{1}{2}} \tag{2.3}$$

Thm Cauchy-Schwarz Inequality **theorem 2.1.1**

For any two vectors $x, y \in \mathbb{R}^n$, the following inequality holds:

$$|(x, y)| = |x^T y| \leq \|\boldsymbol{x}\|_2 \|\boldsymbol{y}\|_2 \tag{2.4}$$

Where strict equality holds if and only if x and y are linearly dependent.

We recall that the scalr product in $\mathbb{R}^n$ can be realyed to the p-norms by the H"older inequality:

$$|(x, y)| \leq \|\boldsymbol{x}\|_p \|\boldsymbol{y}\|_q \qquad \text{Where} \qquad \frac{1}{p} + \frac{1}{q} = 1 \tag{2.5}$$

Thm Norm continuity **theorem 2.1.2**

Any vector norm $\|\cdot\|$ defined on V is a continuous function of its argument, namely, $\forall > 0, \exists C > 0$ such that if $\|\boldsymbol{x} - \hat{\boldsymbol{x}}\| \leq \varepsilon$ then $\|\boldsymbol{x}\| - \|\hat{\boldsymbol{x}}\| \leq C\varepsilon$, for any $x, \hat{x} \in V$.

Thm **theorem 2.1.3**

let $\|\cdot\|$ be a norm of $\mathbb{R}^n$ and $A \in \mathbb{R}^{n \times n}$ be a matrix with n linearly independent columns. Then, the function $\|\cdot\|_{A^2}$ acting from $\mathbb{R}^n$ in to $\mathbb{R}$ defined as:

$$\|\boldsymbol{x}\|_{A^2} = \|\boldsymbol{A}\boldsymbol{x}\| \tag{2.6}$$

is a norm on $\mathbb{R}^n$.

Thm Energy Norm **theorem 2.1.4**

Let $A \in \mathbb{R}^{n \times n}$ is a symmetric positive definite matrix. Then, the *energy norm* is defined as:

$$\|\boldsymbol{x}\|_A = \sqrt{\boldsymbol{x}^T A \boldsymbol{x}} \tag{2.7}$$

Thm Convergence **theorem 2.1.5**

Let $\|\cdot\|$ be a norm in a finite dimensional space V. Then:

$$\lim_{k \to \infty} x^{(k)} = x \iff \lim_{k \to \infty} \|\boldsymbol{x}^{(\boldsymbol{k})} - \boldsymbol{x}\| = 0 \tag{2.8}$$

where $x \in V$ and $x^{(k)}$ is a sequence of vectors in V.

2.2 Matrix Norms

In dealing with a space of matrices, certain special norms are more useful thant the vector norms. These are the *induced matrix norms*, defined in terms of the behavior of a matrix as an operator between its normed domain and range spaces.

Def Matrix Norm **definition 2.2.1**

A *matrix norm* is a mapping $\|\cdot\| : \mathbb{R}^{m\times n} \to \mathbb{R}$ such that:

1. $\|\boldsymbol{A}\| \geq 0 \forall A \in \mathbb{R}^{m\times n}$ and $\|\boldsymbol{A}\| = 0$ if and only if $A = 0$.
2. $\|\boldsymbol{\alpha A}\| = |\alpha| \, \|\boldsymbol{A}\| \; \forall A \in \mathbb{R}^{m\times n}$ and $\alpha \in \mathbb{C}$.
3. $\|\boldsymbol{A} + \boldsymbol{B}\| \leq \|\boldsymbol{A}\| + \|\boldsymbol{B}\| \; \forall A, B \in \mathbb{R}^{m\times n}$(triangular inequality)

Def **definition 2.2.2**

We say that a matrix norm $\|\cdot\|$ is *compatible* or *consistent* with a vector norm $\|\cdot\|$ if:

$$\|\boldsymbol{Ax}\| \leq \|\boldsymbol{A}\| \, \|\boldsymbol{x}\| \qquad \forall x \in \mathbb{R}^n \tag{2.9}$$

More generally, given three norms, all denoted by $\|\cdot\|$, albeit defined on $\mathbb{R}^m, \mathbb{R}^n . \mathbb{R}^{m\times n}$, respectively, we say that they are consistent if if $\forall x \in \mathbb{R}^n, Ax = y \in \mathbb{R}^m$, we have that $\|\boldsymbol{y}\| \leq \|\boldsymbol{A}\| \, \|\boldsymbol{x}\|$.

Def Sub multiplicative **definition 2.2.3**

We say that a matrix norm $\|\cdot\|$ is *sub-multiplicative* if $\forall A \in \mathbb{R}^{n\times m}, \forall B \in \mathbb{R}^{m\times q}$ we have that

$$\|\boldsymbol{AB}\| \leq \|\boldsymbol{A}\| \, \|\boldsymbol{B}\| \tag{2.10}$$

Def Fronebius Norm **definition 2.2.4**

The norm

$$\|\boldsymbol{A}\|_F = \sqrt{\sum_{i=1}^{m}\sum_{j=1}^{n}|a_{ij}|^2} = \sqrt{\mathrm{tr}(AA^H)} \tag{2.11}$$

is a matrix norm called the *Frobenius norm*. And it is compatible with the Euclidean vector norm $\|\cdot\|_2$. Indeed.

$$\|\boldsymbol{Ax}\|_2^2 = \sum_{i=1}^{m}|\sum_{j=1}^{n}a_{ij}x_j|^2 \leq \sum_{i=1}^{m}\sum_{j=1}^{n}|a_{ij}|^2\sum_{j=1}^{n}|x_j|^2 = \|\boldsymbol{A}\|_F^2\|\boldsymbol{x}\|_2^2 \tag{2.12}$$

Thm Induced Matrix Norm **theorem 2.2.1**

Let $\|\cdot\|$ be a vector norm. The function:

$$\|\boldsymbol{A}\| = \sup_{x\neq 0} \frac{\|\boldsymbol{Ax}\|}{\|\boldsymbol{x}\|} \tag{2.13}$$

is a matrix norm called *induced matrix norm or natural matrix norm.*

Proof: Check definition 2.2.1.

1. If $\|\boldsymbol{Ax}\| \geq 0$, then it follows that $\|\boldsymbol{A}\| = \sup_{\|\boldsymbol{x}\|=1}\|\boldsymbol{Ax}\| \geq 0$. Moreover,

$$\|\boldsymbol{A}\| = \sup_{x\neq 0} \frac{\|\boldsymbol{Ax}\|}{\|\boldsymbol{x}\|} = 0 \iff \|\boldsymbol{Ax}\| = 0 \forall x \neq 0 \tag{2.14}$$

and $Ax = 0 \forall x \neq 0$ if and only if $A = 0$; therefore, $\|\boldsymbol{A}\| = 0$ if and only if $A = 0$.

2. Given a scalar α, we have that:

$$\|\boldsymbol{\alpha A}\| = \sup_{x\neq 0} \frac{\|\boldsymbol{\alpha Ax}\|}{\|\boldsymbol{x}\|} = \sup_{x\neq 0} |\alpha| \frac{\|\boldsymbol{Ax}\|}{\|\boldsymbol{x}\|} = |\alpha| \sup_{x\neq 0} \frac{\|\boldsymbol{Ax}\|}{\|\boldsymbol{x}\|} = |\alpha| \ \|\boldsymbol{A}\| \tag{2.15}$$

3. Finally, triangular inequality holds. Indeed, by definition of supremum, if $x \neq 0$ then:

$$\frac{\|\boldsymbol{Ax}\|}{\|\boldsymbol{x}\|} \leq \|\boldsymbol{A}\| \Rightarrow \|\boldsymbol{Ax}\| \leq \|\boldsymbol{A}\| \ \|\boldsymbol{x}\| \tag{2.16}$$

So that, taking x with unit norm, one gets:

$$\|(\boldsymbol{A}+\boldsymbol{B})\boldsymbol{x}\| \leq \|\boldsymbol{Ax}\| + \|\boldsymbol{Bx}\| \leq \|\boldsymbol{A}\| + \|\boldsymbol{B}\| \tag{2.17}$$

from which it follows that $\|\boldsymbol{A}+\boldsymbol{B}\| = \sup_{\|\boldsymbol{x}\|=1}\|(\boldsymbol{A}+\boldsymbol{B})\boldsymbol{x}\| \leq \|\boldsymbol{A}\| + \|\boldsymbol{B}\|$.

Relevant instances of induced matrix norms are the so-called *p-norms*:

$$\|\boldsymbol{A}\|_p = \sup_{\boldsymbol{x}\neq 0} \frac{\|\boldsymbol{Ax}\|_p}{\|\boldsymbol{x}\|_p} \tag{2.18}$$

The 1-norm(*column sum norn*):

$$\|\boldsymbol{A}\|_1 = \max_{j=1,\dots,n} \sum_{i=1}^{m} |a_{ij}| \tag{2.19}$$

The infinity-norm(*row sum norm*):

$$\|\boldsymbol{A}\|_\infty = \max_{i=1,\dots,m} \sum_{j=1}^{n} |a_{ij}| \tag{2.20}$$

Moreover, we have $\|\boldsymbol{A}\|_1 = \|\boldsymbol{A^T}\|_\infty$ and, if A is self-adjoint or real sysmetric, then $\|\boldsymbol{A}\|_1 = \|\boldsymbol{A}\|_\infty$.

A special discussion is deserved by the *2-norm* or *spectral norm* for which the following theorem holds.

Thm Spectral Norm **theorem 2.2.2**

Let $\sigma_1(A)$ be the largest singular value of A. Then, the 2-norm of A is given by:

$$\|\boldsymbol{A}\|_2 = \sqrt{\rho(A^H A)} = \sqrt{\rho(A^H A)} == \sigma_1(A) \tag{2.21}$$

In particular, if A is hermitian (or real and symmetric), then $\|\boldsymbol{A}\|_2 = \rho(A)$.

Proof: Since $A^T A$ is hermitian, there exists a unitary matrix U such that

$$U^H A^H A U = \text{diag}(\mu_1, ..., \mu_n) \tag{2.22}$$

where μ_i are the positive eigenvalues of $A^H A$. Let $y = U^H x$, then:

$$\begin{aligned} \|\boldsymbol{A_2}\| &= \sup_{x \neq 0} \frac{\sqrt{(A^H A x, x)}}{\sqrt{(x,x)}} = \sup_{y \neq 0} \frac{\sqrt{(U^H A^H A U y, y)}}{\sqrt{(y,y)}} \\ &= \sup_{y \neq 0} \frac{\sqrt{\sum_{i=1}^n \mu_i \ |y_i|^2}}{\sqrt{\sum_{i=1}^n |y_i|^2}} = \sqrt{\max_{i=1,...,n}^n |\mu_i|} \end{aligned} \tag{2.23}$$

If A is hermitian, the same considerations as above apply directly to A. Finally, if A is unitary, we have

$$\|\boldsymbol{A}\|_2 = \sup_{x \neq 0} \frac{\|\boldsymbol{Ax}\|_2}{\|\boldsymbol{x}\|_2} = \sup_{x \neq 0} \frac{\|\boldsymbol{x}\|_2}{\|\boldsymbol{x}\|_2} = 1 \tag{2.24}$$

2.3 Sequences and Series of Matrices

A sequence of matrices A^k is said to *converge* to a matrix $A \in \mathbb{R}^{n \times n}$ if

$$\lim_{k \to \infty} \|\boldsymbol{A^k} - \boldsymbol{A}\| = 0 \tag{2.25}$$

The choice of the norm does not influence the result since in $\mathbb{R}^{n \times n}$ all norms are equivalent.

Thm Convergence of Sequences of Matrices **theorem 2.3.1**

Let A be a square matrix; then

$$\lim_{k \to \infty} A^k = 0 \Leftrightarrow \rho(A) < 1 \tag{2.26}$$

3 Principles of Numerical Mathematics

3.1 Well-posedness and Condition Number

Consider the following problem: find x such that:

$$F(x, d) = 0 \tag{3.1}$$

where F is a function of x and d. And three types of problems can be considered:

1. *direct problem*: given F and d, find x;
2. *inverse problem*: given F and x, find d;
3. *identification problem*: given x and d, find F.

Problems Eq. (3.1) are ***well-posed*** if it admits a *unique* solution, and the solution depends continuously on the data.

A problem which does not enjoy the property above is called ill posed or unstable and before undertaking its numerical solution it has to be regularized, that is, it must be suitably transformed into a well-posed problem.

Let D be the set of admissible data, i.e. the set of the values of d in correspondance of which problem Eq. (3.1) admits a unique solution. Continuous dependence on the data means that small perturbations on the data d of D yield "small" changes in the solution x.

Precisely, let $d \in D$ and denoted by δd a perturbation admissible in the sense that $d + \delta d \in D$ and by δx the corresponding change in the solution, in such a way that:

$$F(x + \delta x, d + \delta d) = 0 \tag{3.2}$$

Then, we require that:

$$\begin{gathered} \exists \eta_0 = \eta_0(d) > 0, \exists K_0 = K_0(d) \text{ such that} \\ \text{if } \|\boldsymbol{\delta d}\| \leqslant \eta_0 \text{ then } \|\boldsymbol{\delta x}\| \leqslant K_0 \, \|\boldsymbol{\delta d}\| \end{gathered} \tag{3.3}$$

The norms used for the data and for the solution may not coincide, whenever d and x represent variables of different kinds.

e.g. Wellposedness of Linear Systems **example 3.1.1**

Consider the problem of solving a linear system $Ax = b$. The problem is well-posed if it has below two properties:

1. The problem has a unique solution x, which means that the matrix A is invertible.
2. The solution depends continuously on the data.

The Eq. (3.3) is however more suitable to express in the following the concept of *numerical stability*, that is, the property that small perturbations on the data yield perturbations of the same order on the solution.

3.1.1 Absolute Condition Number

Let δx denote a small perturbation of x, and wirte $\delta f = f(x + \delta x, d) - f(x, d)$. The absolute condition number is then defined as:

$$K_{\text{abs}} = \lim_{\delta \to 0} \sup_{\|\boldsymbol{\delta x}\| \leq \delta} \frac{\|\boldsymbol{\delta f}\|}{\|\boldsymbol{\delta x}\|} \tag{3.4}$$

For most problems, the limit of the supremum in this formula can be interpreted as a supremum over all infinitesimal perturbations δx, and in the interest of readability, we shall generally write the formula simply as

$$K = \sup_{\delta x} \frac{\|\boldsymbol{\delta f}\|}{\|\boldsymbol{\delta x}\|} \tag{3.5}$$

with the understanding that δx and δf are infinitesimal.

If f is differentiable, we can evaluate the absolute condition number by means of the derivative of f. Let $J(x)$ be the matrix whose i, j entry is the partial derivative $\partial f_i / \partial x_j$, evaluated at x. The definition of derivative gives us, $\delta f \approx J(x)\delta x$, with equality in the limit $\|\boldsymbol{\delta x}\| \to 0$. The absolute condition number is then:

$$K = \|\boldsymbol{J}(\boldsymbol{x})\| \tag{3.6}$$

3.1.2 Relative Condition Number

When we are concerned with relative changes, we need the notion of relative condition. The *relative condition number* is defined as:

$$K = \lim_{\delta \to 0} \sup_{\|\boldsymbol{\delta x}\| \leq \delta} \left(\frac{\|\boldsymbol{\delta f}\|}{\|\boldsymbol{f}(\boldsymbol{x})\|} / \frac{\|\boldsymbol{\delta x}\|}{\|\boldsymbol{x}\|} \right) \tag{3.7}$$

or, assuming δx and δf are infinitesimal,

$$K = \sup_{\delta x} \frac{\frac{\|\boldsymbol{\delta f}\|}{\|\boldsymbol{f}(\boldsymbol{x})\|}}{\frac{\|\boldsymbol{\delta x}\|}{\|\boldsymbol{x}\|}} \tag{3.8}$$

If f is differentiable, we can express this equality in terms of the Jacobian matrix $J(x)$, as follows:

$$K = \frac{\|\boldsymbol{J}(\boldsymbol{x})\|}{\|\boldsymbol{f}\boldsymbol{x}\| \ / \ \|\boldsymbol{x}\|} \tag{3.9}$$

Problem Eq. (3.1) is called *ill-conditioned* if $K(d)$ is "big" for any admissible datum d (the precise meaning of "small" and "big" is going to change depending on the considered problem).

3.1.3 Codution of Matrix-Vector Multiplication

Now we come to one of the condition numbers of fundamental importance in numerical linear algebra.

Fix $A \in \mathbb{C}^{m \times n}$ and consider the problem of computing Ax from input x; that is, we are going to determine a condition number corresponding to perturbations of x but not A. Working directly

from the definition of K, with $\|\cdot\|$ denoting an arbitrary vector norm and the corresponding induced matrix norm, we have:

$$K = \sup_{\delta x} \left(\frac{\|\boldsymbol{A}(\boldsymbol{x}+\boldsymbol{\delta x}) - \boldsymbol{A}\boldsymbol{x}\|}{\|\boldsymbol{A}\boldsymbol{x}\|} \right) / \left(\frac{\|\boldsymbol{\delta x}\|}{\|\boldsymbol{x}\|} \right) = \sup_{\delta x} \frac{\|\boldsymbol{A}\boldsymbol{\delta x}\|}{\|\boldsymbol{\delta x}\|} / \frac{\|\boldsymbol{A}\boldsymbol{x}\|}{\|\boldsymbol{x}\|} \tag{3.10}$$

that is,

$$K = \|\boldsymbol{A}\| \, \frac{\|\boldsymbol{x}\|}{\|\boldsymbol{A}\boldsymbol{x}\|} \tag{3.11}$$

This is an exact formula for K, dependent on both A and x.

Suppose A is square and nonsingular. Then we can use the fact that $\|\boldsymbol{x}\| \,/\, \|\boldsymbol{A}\boldsymbol{x}\| \leq \|\boldsymbol{A}^{-1}\|$ to loosen Eq. (3.11) to a bound independent of x:

$$K \leq \|\boldsymbol{A}\| \, \|\boldsymbol{A}^{-1}\| \tag{3.12}$$

Or, one might write this as:

$$k = \alpha \, \|\boldsymbol{A}\| \, \|\boldsymbol{A}^{-1}\| \tag{3.13}$$

with

$$\alpha = \frac{\|\boldsymbol{x}\|}{\|\boldsymbol{A}\boldsymbol{x}\|} / \, \|\boldsymbol{A}^{-1}\| \tag{3.14}$$

If $\|\cdot\| = \|\cdot\|_2$ this will occur whenever x is a multiple of a minimal right singular vector of A.

3.1.4 Condition number of a Matrix

The product $\|\boldsymbol{A}\| \, \|\boldsymbol{A}^{-1}\|$ comes up so often that it has its own name: it is the *condition number* of A:

$$K(A) = \|\boldsymbol{A}\| \, \|\boldsymbol{A}^{-1}\| \tag{3.15}$$

Thus, in this case the term *condition number* is attached to a matrix, not a problem. If $K(A)$ is small, A is said to be *well-conditioned*; if it is large, A is *ill-conditioned*. If A is singular, it is customary to write $K(A) = \infty$.

Note that if $\|\cdot\| = \|\cdot\|^2$, then $\|\boldsymbol{A}\| = \sigma_1$ and $A^{-1} = \frac{1}{\sigma_n}$, where σ_1 and λ_n are the maximum abnd minimum singular value of A. Thus

$$K(A) = \frac{\sigma_1}{\sigma_n} \tag{3.16}$$

In the 2-norm, and it is this formula that is generally used for computing 2-norm condition numbers of matrices.

For a rectangular matrix $A \in \mathbb{C}^{m \times n}$ of full rank, $m \geq n$, the condition numver is defined in terms of the **pseudoinverse**: $K(A) = \|\boldsymbol{A}\| \, \|\boldsymbol{A}^{+}\|$. Since A^+ is motivated by least squares problems, this definition is most useful in the case $\|\cdot\| = \|\cdot\|^2$, where we have

$$K(A) = \frac{\sigma_1}{\sigma_n} \tag{3.17}$$

3.1.5 Condition Number of a System of Equations

Specifically, let us hold b fixed and consider the behavior of the problem $A \to x = A^{-1}b$ when A is perturbed by infinitesimal δA. Then x must change by infinitesimal δx such that:

$$(A + \delta A)(x + \delta x) = 0 \tag{3.18}$$

Using the equality $Ax = b$ and dropping the doubly infinitesimal term $(\delta A)(\delta x)$, we obtain $(\sigma A)x + A(\sigma)x = 0$. that is, $\sigma x = -A^{-1}(\sigma A)x$. This equation implies $\|\boldsymbol{\sigma x}\| \leq \|\boldsymbol{A^{-1}}\| \, \|\boldsymbol{\sigma A}\| \, \|\boldsymbol{x}\|$, or equivalently:

$$\frac{\sigma x}{\|\boldsymbol{x}\|} / \frac{\sigma A}{\|\boldsymbol{A}\|} \leq \|\boldsymbol{A^{-1}}\| \, \|\boldsymbol{A}\| = K(A) \tag{3.19}$$

Thm **theorem 3.1.1**

Let b be fixed and consider the problem of computing $x = A^{-1}b$, where A is square and nonsingular. The condition number of this problem with respect to perturbations in A is

$$K(A) = \|\boldsymbol{A}\| \, \|\boldsymbol{A^{-1}}\| \tag{3.20}$$

3.2 Stability of Numerical Methods

We shall henceforth suppose the problem Eq. (3.1) to be well-posed and a numerical method for the approximate solution of Eq. (3.1) will consist, in general, of a sequence of approximate problems:

$$F(x_n, d_n) = 0 \tag{3.21}$$

depending on a certain parameter n (to be defined case by case). The undetstood expectation is that $x_n \to x$ as $n \to \infty$, that is, the sequence of approximate solutions **converges** to the exact solution.

For that, it is necessary that $d_n \to d$ and $F_n \to F$, as $n \to \infty$. Precisely, if the datum d of Eq. (3.1) is admissible for F_n, we say that Eq. (3.21) is consistent if:

$$F_{n(x,d)} = F_{n(x,d)} - F(x, d) \to 0 \text{ for } n \to \infty \tag{3.22}$$

3.2.1 Relations between Stability and Covergence

The concepts of stability and convergence are strongly connected.

Thm **theorem 3.2.1**

If problem Eq. (3.1) is well-posed, a *necessary* condition in order for the numerical problem Eq. (3.21) to be convergent is that it is stable.

4 Nolinear Equations

4.1 The bisection method

Let f be a continuous function in $[a, b]$ which satisfies $f(a)f(b) < 0$. Then **necessarily** f has at least one zero in (a, b).

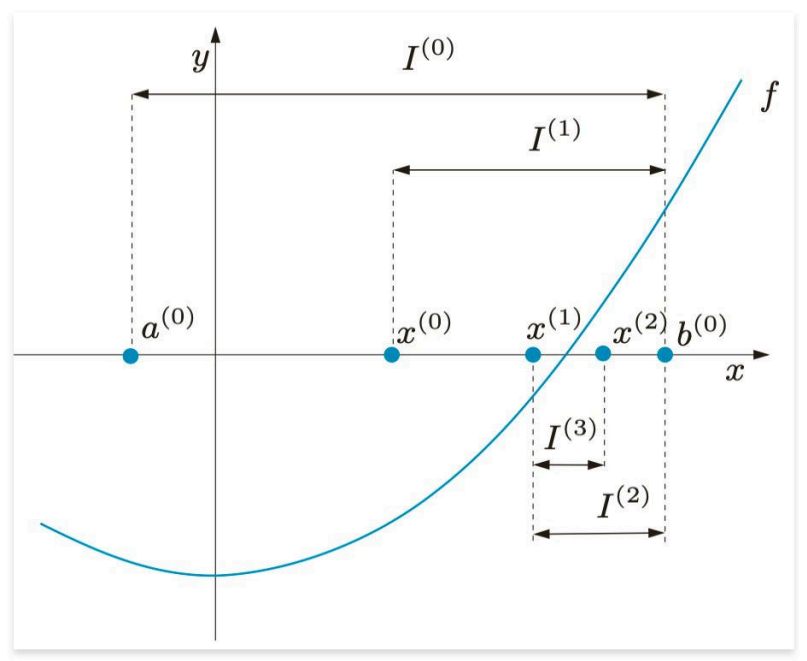


Figure 4.1: Iterations of the bisection method

The strategy of the bisection method is to havle the given interval and select that subinterval where f fatures a sign change.

4.1.1 Chord Method

The bisection method is a very robust method, but it is also very slow. The chord method is a faster method, but it is also less robust. The chord method is based on the idea of replacing the curve $(x, f(x))$ by the chord that joins the points $(a, f(a))$ and $(b, f(b))$.

The equation of the chord is:

$$x^{(k+1)} = x^{(k)} - f\big(x^{(k)}\big)\frac{b-a}{f(b)-f(a)}, k \geq 0 \tag{4.1}$$

4.2 The Newton method

The sign of the given function f at the endpoints of the subintervals is the only information exploited by the bisection method. A more efficient method can be constructed by exploiting the values attained by f and its derivative. In that case,

$$y(x) = f\big(x^{(k)}\big) + f'\big(x^{(k)}\big)\big(x - x^{(k)}\big) \tag{4.2}$$

provides the equation of the tangent to the curve $(x, f(x))$ at the point $x^{(k)}$.

If we pretend that $x^{(k+1)}$ is such that $y\big(x^{(k+1)}\big) = 0$, we obtain:

$$x^{(k+1)} = x^{(k)} - \frac{f\left(x^{(k)}\right)}{f'\left(x^{(k)}\right)}, k \geq 0 \tag{4.3}$$

provided that $f'\left(x^{(k)}\right) \neq 0$. This formula allows us to compute a sequence of values $x^{(k)}$ starting from an initial guess $x^{(0)}$. This method is known as Newton's method and corresponds to computing the zeros of f by locally replacing f by its tangent.

The Newton method in general does not converge for all possible choices of $x^{(0)}$, but only for those value of $x^{(0)}$ which are **sufficiently close** to α. In order to compute α, one should start from a value sufficiently close to α.

In practice, the initial value $x^{(0)}$ can be obtained by resorting to a few iterations of the bisection method of, through an investigation of the graph of f. If $x^{(0)}$ is properly closen and α is a simple zero ($f'(\alpha) \neq 0$), then the Newton method converges **quadratically** to α. If $f'(\alpha) = 0$, the method converges at most linearly.

4.2.1 Stopping criteria

In practice, one requires an approximation of α up to a prescribed tolerance ε. Thus the iterations can be terminated at the smallest value of $k_{\min}$ for which the following inequality holds:

$$|e^{(k_{\min})}| = |x^{(k_{\min})} - \alpha| < \varepsilon \tag{4.4}$$

Alternatively, one could use a test on the *residual* at step k, $r^{(k)} = f\left(x^{(k)}\right)$.

Precisely, we could stop the iteration at the first $k_{\min}$ for which

$$|r^{(k_{\min})}| = |f\left(x^{(k_{\min})}\right)| < \varepsilon \tag{4.5}$$

The test on the residual is satisfactory only when $|f'(x)| \simeq 1$ in a neighborhood of I_α of the zero α. Otherwise, it will produce an over estimation of the error if $|f'(x)| \gg 1$ and an under estimation if $|f'(x)| \ll 1$.

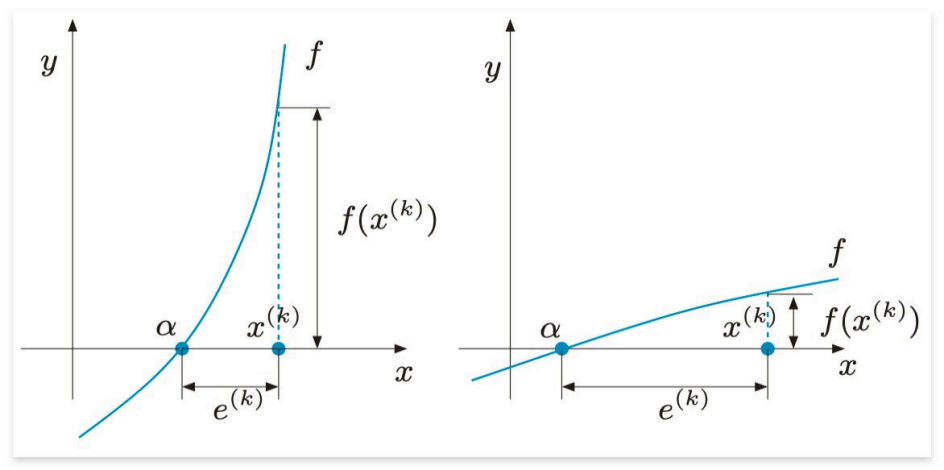


Figure 4.2: Two situations in which the residual is a poor error estimator: $|f'(x)| \gg 1$, $|f'(x)| \ll 1$

4.2.2 Modified Newton Method

If the root α is not simple, the Newton method converges with order 1. If m is the multiplicity of the root, then the modification

$$x^{(k+1)} = x^{(k)} - m\frac{f\left(x^{(k)}\right)}{f'\left(x^{(k)}\right)},\ k \geq 0,\ f'\left(x^{(k)}\right) \neq 0 \tag{4.6}$$

allow us to recover the second order of convergence.

4.3 Fixed Point Iterations

Given a function $g : [a, b] \to \mathbb{R}$, find $\alpha \in [a, b]$ such that

$$\alpha = g(\alpha) \tag{4.7}$$

If such an α exists it will be called a *fixed point* of g. The fixed point iteration is defined as:

$$x^{(k+1)} = g\left(x^{(k)}\right), k \geq 0 \tag{4.8}$$

where $x^{(0)}$ is an initial guess.

Solving the fixed point problem $x = g(x)$ is equivalent to solving the system

$$\begin{cases} y = x \\ y = g(x) \end{cases} \tag{4.9}$$

to determine the intersections between $g(x)$ and the bisector line $y = x$ of the first and the third quadrants.

Thm Convergence of Fixed Point Iterations **theorem 4.3.1**

Assume that the iteration function in the fixed point iteration is satisfies the following properties:

1. $g(x) \in [a, b]$ for all $x \in [a, b]$;
2. g is differentiable in $[a, b]$;
3. $\exists K < 1$ such that $|g'(x)| < K$ for all $x \in [a, b]$.

Then g has a unique fixed point $\alpha \in [a, b]$ and the sequence defined in the fixed point iteration converges to α for any initial guess $x^{(0)}$. Moreover

$$\lim_{k\to\infty} x^{(k)} = \frac{x^{(k+1)} - \alpha}{x^{(k)} - \alpha} = g'(\alpha) \tag{4.10}$$

Let α be a fixed point of the a function g which is continuous and differentiable in a neighborhood of α.

- if $|g'(\alpha)| > 1$, then the sequence $x^{(k+1)} = g\left(x^{(k)}\right)$ will not converge to α.
- if $|g'(\alpha)| = 1$, then no general conclusion can be drawn both convergence and divergence become possibile.

Let's see an example of the fixed point iteration.

e.g. Convergence of Fixed Point Iterations **example 4.3.1**

The function $g(x) = \cos(x)$ satisfies all the assumptions theorem 4.3.1. Indeed, $|g'(\alpha)| = |\sin(\alpha)| \simeq 0.67 < 1$, and thus by continuity there exists a neighborhood I_α of α such that $|g'(x)| < 1$ for all $x \in I_\alpha$.

The function $g(x) = x^2 - 1$ has two fixed points $\alpha_\pm = \frac{1+\sqrt{5}}{2}$. However,it does not satisfy the assumption for either since $g'(\alpha_\pm) = |1 \pm \sqrt{5}| > 1$. The corresponding fixed point iterations will not converge.

Thm order p convergence **theorem 4.3.2**

Assume that all hypothese of theorem 4.3.1 are satisfied. In addition assume that

$$g^{(i)}(\alpha) = 0, \quad g^{(i)}(\alpha) \neq 0, \quad i = 1, ..., p-1 \tag{4.11}$$

Then the fixed point iterations converge with order p and

$$\lim_{k\to\infty} \frac{x^{(k+1)} - \alpha}{(x^{(k)} - \alpha)^p} = \frac{g^{(p)}(\alpha)}{p!} \tag{4.12}$$

Consider the limit of the above-mentioned quantity for two consecutive steps and, for $k \to \infty$, analyze the corresponding error defining $e^{(k)} = x^{(k)} - \alpha$:

$$\frac{e^{(k+1)}}{(e^{(k)})^p} = \frac{e^{(k)}}{(e^{(k-1)})^p} \tag{4.13}$$

so that

$$\frac{e^{(k+1)}}{e^{(k)}} = \left(\frac{e^{(k)}}{e^{(k-1)}}\right)^p \tag{4.14}$$

and applying the logarithm

$$p = \frac{\log\left(\frac{e^{(k+1)}}{e^{(k)}}\right)}{\log\left(\frac{e^{(k)}}{e^{(k-1)}}\right)} = \frac{\log(e^{(k+1)}) - \log(e^{(k)})}{\log(e^{(k)}) - \log(e^{(k-1)})} \tag{4.15}$$

4.3.1 Stopping criteria

In general, fixed point iterations are terminated when the absolute value of the difference between two consecutive iterates is less than a prescribed tolerance ε.

5 Approximation of functions and data

Approximating a function f consists of replacing it by another function $\tilde{f}$ of simpler form that may be used as its surrogate. For example, instead of computing $\int_a^b f(x)dx$, one carries out the axcat computation of $\int_a^b \tilde{f}(x)dx$. In these cases we aim at constructing a continuous function $\tilde{f}$ that could represent the empirical law which is behind the finite set of data.

5.1 Interpolation

In several applications it may happen that a function is known only through its values at some given points. We are therefore facing a case where $n+1$ couples $\{x_i, y_i\}, i = 0, ..., n$, are given; the points x_i are all distinct and are called *nodes*.

In such a situation it seems natural to require the approximate function $\tilde{f}$ to satisfy the set of relations:

$$\tilde{f}(x_i) = y_i, i = 0, ..., n \tag{5.1}$$

Such $\tilde{f}$ is called *interpolants* of the set of data and Eq. (5.1) are the interpolation conditions.

5.2 Polynomial Interpolation

Let us focus on the polynomial interpolation. The following result holds:

Thm Weierstrass Approximation Theorem **theorem 5.2.1**

Let f be a continuous function in $[a, b]$. Then, for any $\varepsilon > 0$, there exists a polynomial $P_{n(x)}$ of degree $\leq n$ such that

$$|f(x) - P_{n(x)}| < \varepsilon, \quad \forall x \in [a, b] \tag{5.2}$$

Thm Existence and Uniqueness of the Interpolant **theorem 5.2.2**

For any set of couples $\{x_i, y_i\}, i = 0, ..., n$, with distinct nodes x_i, there exists a unique polynomial of degree less than or equal to n, which we indicate by Π_n and call interpolating polynomial of the values y_i at the nodes x_i, such that

$$P_n(x_i) = y_i, \quad i = 0, ..., n \tag{5.3}$$

In the case where the $\{y_i, i = 0, ..., n\}$ represent the values of a continuous function f, P_n is called interpolating polynomial of f and will be denoted by $P_n f$.

5.3 Lagrange Interpolation

In order to obtain an expression for P, we start from a very special case where y_i vanishes for all i apart from $i = k$ (for a fixed k) for which $y_k = 1$. Then setting $L_k(x) = P_n(x)$, we must have:

$$L_k(x_i) = \delta_{ik} = \begin{cases} 1 & i = k \\ 0 & i \neq k \end{cases}, \quad i = 0, ..., n \tag{5.4}$$

The functions $L_{k,n}(x)$ have the following expression:

$$L_{k,n}(x) = \prod_{i=0, i\neq k}^{n} \frac{x - x_i}{x_k - x_i}, n \geq 1, \quad L_{0,0}(x) = 1 \tag{5.5}$$

$P(x) = \sum_{k=0}^{n} L_{k,n}(x) f(x_k)$ is a polynomial of degree $\leq n$ that interpolates f at $x_0, ..., x_n$. This representation is called the Lagrange form of the interpolating polynomial.

5.4 Trigonometry Interpolation

We want to approximate a periodic function $f : [0, 2\pi] \to \mathbb{C}$ one satisfying $f(0) = f(2\pi)$, by a trigonometry polynomial $\tilde{f}$ which interpolates f at $n+1$ points $x_i = \frac{2\pi i}{n+1}, i = 0, ..., n$. The constant step size is

$$h = \frac{2\pi}{n+1} \tag{5.6}$$

The Lagrange interpolator may not approximate well far from the nodes, as it is a polynomial of degree n and generally not periodic.

In particular, if n is even, $\tilde{f}$ will have the form

$$\tilde{f}(x) = \frac{a_0}{2} + \sum_{k=1}^{M} [a_k \cos(kx) + b_k \sin(kx)], \quad M = \frac{n}{2} \tag{5.7}$$

If n is odd

$$\tilde{f}(x) = \frac{a_0}{2} + \sum_{k=1}^{M} [a_k \cos(kx) + b_k \sin(kx)] + a_{M+1} \cos((M+1)x), \quad M = \frac{n-1}{2} \tag{5.8}$$

5.5 Least Squares Approximation

As already noticed, a Lagrange interpolation does not guarantee a better approximation of a given function when the polynomial degree gets large. This problem can be overcome by composite interpolation (such as piecewise linear polynomials or splines). However, neither are suitable to extrapolate information from the available data, that is, to generate new values at points lying outside the interval where interpolation nodes are given.

Assume that the data $\{(x_i, y_i), i = 0, ..., n\}$. For a given integer $m \geq 1$ we look for a polynomial $\tilde{f} \in \mathbb{P}_m$ which satisfies the inequality

$$\sum_{i=0}^{n} \left[y_i - \tilde{f}(x_i)\right]^2 \leq \sum_{i=0}^{n} \left[y_i - p_m(x_i)\right]^2 \tag{5.9}$$

for every polynomial $p_m \in \mathbb{P}_m$. Should it exist, $\tilde{f}$ will be called the *least squares approximation* in $\mathbb{P}_m$ of the set of data $\{(x_i, y_i), i = 0, ..., n\}$. Unless $m \geq n$, in general it will not be possible to guarantee that $\tilde{f}(x_i) = y_i$ for all $i = 0, ..., n$.

Setting

$$\tilde{f}(x) = a_0 + a_1 x + ... + a_m x^m \tag{5.10}$$

where the coefficients $a_0, ..., a_m$ are unknown, the least squares approximation can be restated as follows: find the coefficients $a_0, ..., a_m$ such that

$$\Phi(a_0, a_1, ..., a_m) = \min_{b_i, i=0,...,m} \Phi(b_0, b_1, ..., b_m) \tag{5.11}$$

where

$$\Phi(b_0, b_1, ..., b_m) = \sum_{i=0}^{n} [y_i - (b_0 + b_1 x_i + ... + b_m x_i^m)]^2 \tag{5.12}$$

We solve this problem in the special case when $m = 1$. Since

$$\Phi(b_0, b_1) = \sum_{i=0}^{n} [y_i^2 + b_0^2 + b_1^2 x_i^2 + 2b_0 b_1 x_i - 2b_0 y_i - 2b_1 x_i y_i^2] \tag{5.13}$$

the graph of Φ is a convex paraboloid. The point (a_0, a_1) at which Φ attains its minimum satisfies the conditions

$$\frac{\partial \Phi}{\partial b_0}(a_0, a_1) = 0, \quad \frac{\partial \Phi}{\partial b_1}(a_0, a_1) = 0 \tag{5.14}$$

By explicitly computing the two partial derivatives we obtain

$$\sum_{i=0}^{n} [a_0 + a_1 x_i - y_i] = 0, \quad \sum_{i=0}^{n} [x_i(a_0 + a_1 x_i) - x_i y_i] = 0 \tag{5.15}$$

which is a system of two equations for the two unknowns a_0 and a_1:

$$\begin{aligned} a_0(n+1) + a_1 \sum_{i=0}^{n} x_i &= \sum_{i=0}^{n} y_i, \\ a_0 \sum_{i=0}^{n} x_i + a_1 \sum_{i=0}^{n} x_i^2 &= \sum_{i=0}^{n} x_i y_i \end{aligned} \tag{5.16}$$

Setting $D = (n+1)\sum_{i=0}^{n} x_i^2 - \left(\sum_{i=0}^{n} x_i\right)^2$, the solution reads:

$$\begin{aligned} a_0 &= \frac{1}{D}\left(\sum_{i=0}^{n} x_i^2 \sum_{i=0}^{n} y_i - \sum_{i=0}^{n} x_i \sum_{i=0}^{n} x_i y_i\right), \\ a_1 &= \frac{1}{D}\left((n+1)\sum_{i=0}^{n} x_i y_i - \sum_{i=0}^{n} x_i \sum_{i=0}^{n} y_i\right) \end{aligned} \tag{5.17}$$

The corresponding polynomial $\tilde{f}(x) = a_0 + a_1 x$ is known as the *least squares straight line*, or *regression line.*

6 Numerical differentiation and integration

Concerning integration, quite often for a generic function **it is not possible to find a primitive in an explicit form**. Even when a primitive is known, its use might not be easy. In all these situations it is necessary to consider numerical methods in order to obtain an approximate value of the quantity of interest, independently of how difficult is the function to integrate or differentiate.

6.1 Approximation of function derivatives

Consider a function $f : [a, b] \to \mathbb{R}$ continuously differentiable in $[a, b]$. We seek an approximation of the first derivative of f at a generic point $\bar{x}$ in (a, b).

For h sufficiently small and positive, we can assume that the quantity

$$(\sigma_+ f)(\bar{x}) = \frac{f(\bar{x} + h) - f(\bar{x})}{h} \tag{6.1}$$

is an approximation of $f's(\bar{x})$ which is called the *forward finite difference*. To estimate the error, it suffices to expand f in a Taylor series; if $f \in C^2(a, b)$, we have

$$f(\bar{x} + h) = f(\bar{x}) + hf's(\bar{x}) + \frac{h^2}{2} f''(\xi), \quad \xi \in (\bar{x}, \bar{x} + h) \tag{6.2}$$

where ξ is a suitable point in the interval $(\bar{x}, \bar{x} + h)$. Therefore

$$(\sigma_+ f)(\bar{x}) = \left(f'(\bar{x}) + \frac{h}{2} f''(\xi) \right) \tag{6.3}$$

and thus $(\sigma_+ f)(\bar{x})$ provides a first-order approximation to $f'(\bar{x})$ with respect to h. Still assuming $f \in C^2(a, b)$, with a similar procedure we can derive from the Taylor expansion

$$f(\bar{x} - h) = f(\bar{x}) - hf's(\bar{x}) + \frac{h^2}{2} f''(\eta), \quad \eta \in (\bar{x} - h, \bar{x}) \tag{6.4}$$

The backward finite difference is defined as

$$(\sigma_- f)(\bar{x}) = \frac{f(\bar{x}) - f(\bar{x} - h)}{h} \tag{6.5}$$

Finally, we introduce the *centered finite difference* formula

$$(\sigma f)(\bar{x}) = \frac{f(\bar{x} + h) - f(\bar{x} - h)}{2h} \tag{6.6}$$

6.2 Numerical integration

Numerical methods suitable for approximating the integral

$$I(f) = \int_a^b f(x)dx \tag{6.7}$$

where f is an arbitrary continuous function in $[a, b]$.

6.2.1 Midpoint formula

A simple procedure to approximate $I(f)$ can be devised by partitioning the interval $[a, b]$ into

subintervals $I_k = [x_{k-1}, x_k], k = 1, ..., M$, with $x_k = a + kH, k = 0, ..., M$ and $H = (b-a)/M$. Since

$$I(f) = \sum_{k=1}^{M} \int_{I_k} f(x)dx \tag{6.8}$$

on each sub-interval I_k we can approximate the exact integral of f by that of a polynomial $\bar{f}$ approximating f on I_k. The simplest solution consists in choosing $\bar{f}$ as the constant polynomial interpolating f at the middle point of I_k:

$$\bar{x} = \frac{x_{k-1} + x_k}{2} \tag{6.9}$$

In such a way we obtain the composite midpoint quadrature formula

$$I^c_{mp}(f) = H \sum_{k=1}^{M} f(\bar{x_k}) \tag{6.10}$$

The symbol mp stands for midpoint, while c stands for composite. This formula is second-order accurate with respect to H. More precisely, if f is continuously differentiable up to its second derivative in $[a, b]$, we have

$$I(f) - I^c_{mp}(f) = -\frac{H^2}{24}(b-a)f''(\xi), \quad \xi \in (a, b) \tag{6.11}$$

The classical *midpoint formula* (or rectangle formula) is obtained by taking $M = 1$ in, using the midpoint rule directly on the interval $[a, b]$:

$$I_{mp}(f) = (b-a)f\left(\frac{a+b}{2}\right) \tag{6.12}$$

The error is now given by

$$I(f) - I_{mp}(f) = \frac{(b-a)^3}{24}f''(\xi), \quad \xi \in (a, b) \tag{6.13}$$

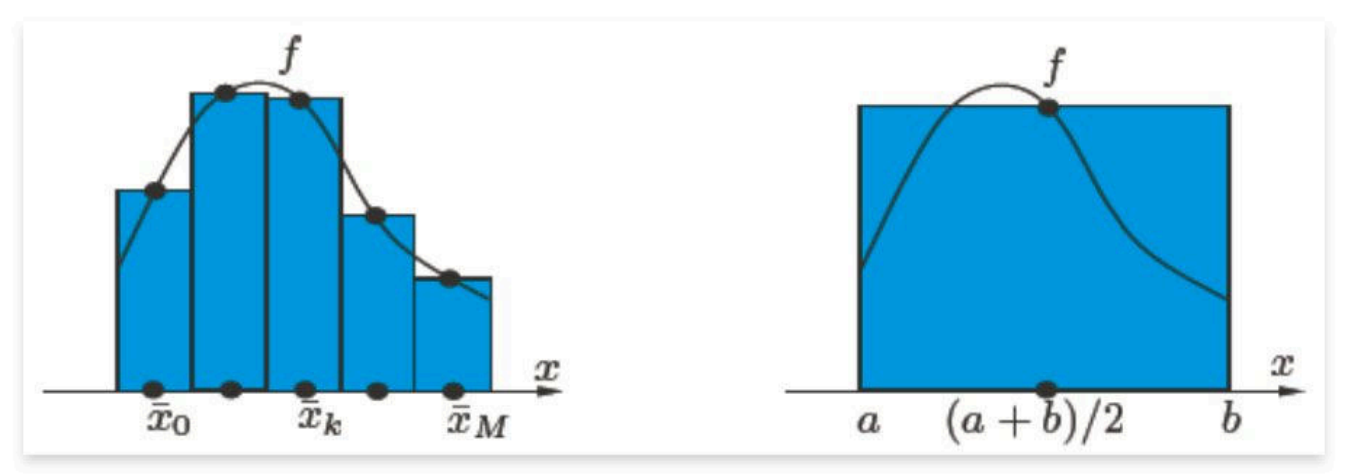


Figure 6.3: The composite midpoint formula (left); the midpoint formula (right)

6.2.2 Trapezoidal formula

Another formula can be obtained by replacing f on I_k by the linear polynomial interpolating f at the nodes x_{k-1} and x_k. This yields

$$\begin{aligned} I_t^c(f) &= \frac{H}{2}\sum_{k=1}^{M}[f(x_k)+f(x_{k-1})] \\ &= \frac{H}{2}[f(a)+f(b)] + H\sum_{k=1}^{M-1} f(x_k) \end{aligned} \tag{6.14}$$

This formula is called the *composite trapezoidal formula*, and is second-order accurate with respect to H. The error is given by

$$I(f) - I_t^c(f) = -\frac{H^2}{12}(b-a)f''(\xi) \tag{6.15}$$

for the quadrature error for a suitable point $\xi \in (a,b)$, provided that $f \in C^2([a,b])$. When $M = 1$, we obtain

$$I_t(f) = \frac{b-a}{2}[f(a)+f(b)] \tag{6.16}$$

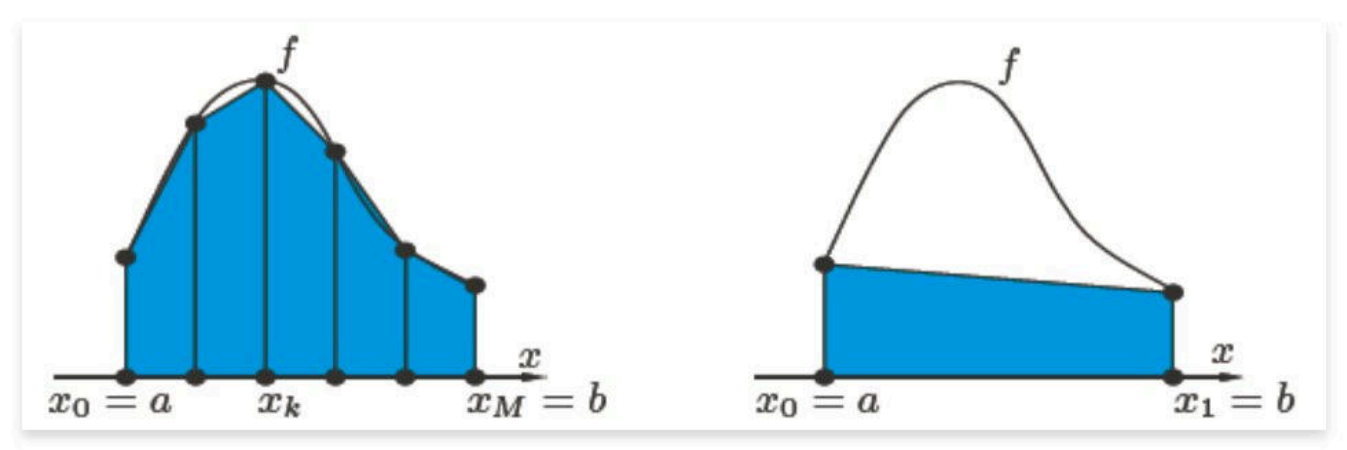


Figure 6.4: The composite trapezoidal formula (left); the trapezoidal formula (right)

which is called the trapezoidal formula because of its geometrical interpretation. The error induced is given by

$$I(f) - I_t(f) = \frac{(b-a)^3}{12}f''(\xi), \quad \xi \in (a,b) \tag{6.17}$$

6.2.3 Simpson formula

The Simpson formula can be obtained by replacing the integral of f over each I_k by that of its interpolating polynomial of degree 2 at the nodes $x_{k-1}, \bar{x}_k = (x_{k-1}+x_k)/2$ and x_k,

$$\begin{aligned} P_2 f(x) = &\frac{2(x-\bar{x}_k)(x-x_k)}{H^2}f(x_{k-1}) \\ &+\frac{4(x_{k-1}-x)(x-x_k)}{H^2}f(\bar{x}_k) + \frac{2(x-\bar{x}_k)(x-x_{k-1})}{H^2}f(x_k) \end{aligned} \tag{6.18}$$

The resulting formula is called the *composite Simpson quadrature formula*, and reads

$$I_s^c(f) = \frac{H}{6}\sum_{k=1}^{M}[f(x_{k-1}) + 4f(\bar{x}_k) + f(x_k)] \tag{6.19}$$

One can prove that it induces the error

$$I(f) - I_s^c(f) = -\frac{b-a}{180}\frac{H^4}{16}f^{(4)}(\xi) \tag{6.20}$$

where ξ is a suitable point in $[a,b]$, provided that $f \in C^4([a,b])$. It is therefore fourth-order accurate with respect to H. When $M = 1$, we obtain the *Simpson formula*

$$I_s(f) = \frac{b-a}{6}\left[f(a) + 4f\left(\frac{a+b}{2}\right) + f(b)\right] \tag{6.21}$$

The error is now given by

$$I(f) - I_s(f) = -\frac{1}{16}\frac{(b-a)^5}{180}f^{(4)}(\xi) \tag{6.22}$$

for a suitable $\xi \in [a,b]$.

7 Ordinary differential equations

A differential equation is an equation involving one or more derivatives of an unknown function. If all derivatives are taken with respect to a single independent variable we call it an *ordinary differential equation*, whereas we have a *partial differential equation* when partial derivatives are present.

The differential equation (ordinary or partial) has order p if p is the maximum order of differentiation that is present.

- A numerical method is called a ***one-step method*** if the numerical solution at the next node t_{n+1} depends only on the information related to the previous node t_n.
- A method is called ***multi-step*** if the numerical solution at the next node t_{n+1} depends also on the information related to the previous nodes $t_{n-1}, t_{n-2}, ..., t_{n-m}$.

7.1 The Cauchy problem

An ordinary differential equation in general admits an infinite number of solutions. In order to fix one of them we must impose a further condition which prescribes the value taken by this solution at a given point of the integration interval.

We will therefore consider the solution of the so-called *Cauchy problem* which takes the following form:

Find $y : I \longrightarrow \mathbb{R}$ such that

$$\begin{cases} y'(t) = f(t, y(t)) & \forall x \in I \\ y(t_0) = y_0 \end{cases} \tag{7.1}$$

where I is an interval of $\mathbb{R}$, $f : I \times \mathbb{R} \longrightarrow \mathbb{R}$ is a given function and y' denotes the derivative of y with respect to t. Finally, t_0 is a point of I and y_0 a given value which is called the *initial data*.

Thm Existence and Uniqueness of the Solution **theorem 7.1.1**

Assume that the function $f(t, y)$ is

1. continuous with respect to both arguments
2. *Lipschitz-continuous* with respect to its second argument, that is, there exists a positive constant L such that

$$|f(t, y_1) - f(t, y_2)| \leq L\ |y_1 - y_2|, \quad \forall t \in I, y_1, y_2 \in \mathbb{R} \tag{7.2}$$

Then the soultion $y = y(t)$ of the Cauchy problem exists, is unique and belongs to $C^1(I)$.

Unfortunately, explicit solutions are available only for very special types of ordinary differential equations. For all these reasons, we seek *numerical methods* capable of approximating the solution of every family of ordinary differential equations for which solutions do exist.

The common strategy of all these methods consists of subdividing the integration interval $I = [t_0, T]$, with $T < +\infty$, into N_h intervals of length $h = \frac{T-t_0}{N_h}$; h is called the *discretization step*. Then, at each node $t_{n(0\leq n\leq N_h-1)}$ we seek the unknow value u_n which approximates $y_n = y(t_n)$. The set of values $\left\{u_0 = y_0, u_1, ..., u_{N_h}\right\}$ is our *numerical solution*.

7.2 Euler methods

A classical method, the *forward Euler* method, generates the numerical solution as follows:

$$u_{n+1} = u_n + hf_n, \quad n = 0, ..., N_h - 1 \tag{7.3}$$

where we have used the shorthand notation $f_n = f(t_n, u_n)$. This method is obtained by considering the differential equation Eq. (7.1) at every node $t_n, n = 1, ..., N_h$ and replacing the exact derivative $y'(t_n)$ by means of the incremental ratio.

In a similar way, using this time the incremental ratio to approximate $y'(t_{n+1})$, we obtain the *backward Euler* method:

$$u_{n+1} = u_n + hf_{n+1}, \quad n = 0, ..., N_h - 1 \tag{7.4}$$

Both methods provide an instance of a *one-step* method since for computing the numerical solution u_{n+1} at the node t_{n+1} we only need the information related to the previous node t_n. More precisely, in the forward Euler method u_{n+1} depends exclusively on the value u_n previously computed, whereas in the backward Euler method it depends also on itself through the value f_{n+1}. For this reason the first method is called the *explicit* Euler method and the second the *implicit* Euler method.

Thus, implicit methods are more costly than explicit methods, since at every time-level t_{n+1} we must solve a nonlinear problem to compute u_{n+1}. However, we will see that **implicit methods enjoy better stability properties than explicit ones**.

7.2.1 Convergence analysis

A numerical method is *convergent* if

$$\forall n = 0, ..., N_h, \quad |y_n - u_n| \leq C(h) \tag{7.5}$$

where $C(h)$ is infinitesimal with respect to h when h tends to zero. If $C(h) = O(h^p)$ for some $p > 0$, then we say that the method converges with order p. In order to verify that the forward Euler method converges, we write the error as follows:

$$e_n = y_n - u_n = (y_n - u_n^*) + (u_n^* - u_n) \tag{7.6}$$

where

$$u_n^* = y_{n-1} + hf(t_{n-1}, u_{n-1}) \tag{7.7}$$

denotes the numerical solution at time t_n which we would obtain starting from the exact solution at time t_{n-1}.

The term $y_n - u_n^*$ represents the error produced by a single step of the forward Euler method, whereas the term $u_n^* - u_n$ represents the propagation from t_{n-1} to t_n of the error accumulated

at the previous time-level t_{n-1}. The method converges provided both terms tend to zero as $h \to 0$. Assuming that the second order derivative of y exists and is continuous, we find

$$y_n - u_n^* = \frac{h^2}{2} y''(\xi_n), \text{for a suitable } \xi_n \in (t_{n-1}, t_n) \tag{7.8}$$

The quantity

$$\tau_n(h) = \frac{y_n - u_n^*}{h} \tag{7.9}$$

is named ***local truncation error*** of the forward Euler method. More in general, the local truncation error of a given method represents the error that would be generated by forcing the exact solution to satisfy that specific numerical scheme, whereas the ***global truncation error*** is defined as

$$\tau(h) = \max_{0,\dots,N_h} |\tau_n(h)| \tag{7.10}$$

The truncation error for the forward Euler method takes the following form:

$$\tau(h) = \frac{Mh}{2} \tag{7.11}$$

where $M = \max_{t \in [t_0, T]} |y''(t)|$.

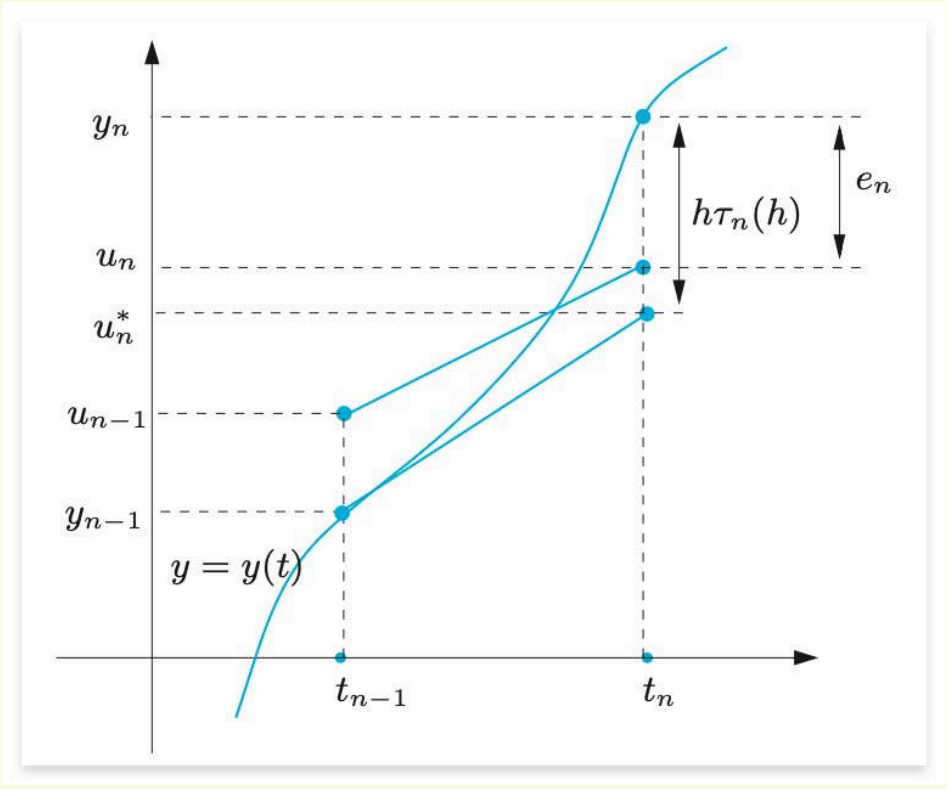


Figure 7.5: Geometrical representation of a step of the forward Euler method

From Eq. (7.8) we deduce that $\lim_{h \to 0} \tau(h) = 0$, and a method for which this happens is said to be *consistent*. Further, we say that it is consistent with order p if $\tau(h) = O(h^p)$ for a suitable integer $p \geq 1$.

Thm Convergence of the Forward Euler Method **theorem 7.2.1**

Assume that the function $f(t, y)$ is Lipschitz-continuous with respect to its second argument. Then the forward Euler method is convergent with order 1.

$$|e_n| \leq \frac{e^{L(t_n - t_0)} - 1}{L} \frac{M}{2} h \tag{7.12}$$

7.3 The Crank-Nicolson method

Adding together the generic steps of the forward and backward Euler methods we find the so-called *Crank-Nicolson method*

$$u_{n+1} = u_n + \frac{h}{2}[f_n + f_{n+1}], \ n = 0, ..., N_h - 1 \tag{7.13}$$

If we assume that $y \in C^3$, we deduce that

$$\tau_n(h) = -\frac{h^2}{12} y'''(\xi_n) \text{ for a suitable } \xi_n \in (t_{n-1}, t_n) \tag{7.14}$$

Thus the Crank-Nicolson method is consistent with order 2. Its local truncation error tends to 0 as h^2. The Crank-Nicolson method is convergent with order 2 with respect to h.

7.4 Zero-stability

There is a concept of stability, called zero-stability, which guarantees that, in a fixed bounded interval, small perturbations of data yield bounded perturbations of the numerical solution when $h \to 0$.

7.5 Stability on unbounded intervals

There are several situations in which the Cauchy problem needs to be integrated on very large (virtually infinite) time intervals. In this case, even if h is fixed, N_h tends to infinity, and then results become meaningless. We are therefore interested in methods that are able to approximate the solution for arbitrarily long time-intervals, even with a step-size h relatively "large".

Consider the following *model problem*

$$\begin{cases} y'(t) = \lambda y(t) & t \in (0, \infty) \\ y(0) = 1 \end{cases} \tag{7.15}$$

where λ is a negative real number. The exact solution of this problem is $y(t) = e^{\lambda t}$, which tends to 0 as t tends to infinity. Applying the forward Euler method, we find that

$$u_0 = 1, \quad u_{n+1} = u_n(1 + \lambda h) = (1 + \lambda h)^{n+1}, n \geq 0 \tag{7.16}$$

Thus $\lim_{n\to\infty} u_n = 0$ iff

$$-1 < 1 + h\lambda < 1 \longrightarrow h < \frac{2}{|\lambda|} \tag{7.17}$$

This condition expresses the requirement that, for *fixed* h, the numerical solution should reproduce the behavior of the exact solution when t_n tends to infinity. If $h > \frac{2}{|\lambda|}$, then $\lim_{n\to\infty} |u_n| = +\infty$.

Thus Eq. (7.17) is a stability condition. The property that $\lim_{n\to\infty} u_n = 0$ is called *absolute stability.*

In contrast to the forward Euler method, neither the backward Euler method nor the Crank-Nicolson method require limitations on h for absolute stability. In fact, with the backward Euler method we obtain $u_{n+1} = u_n + \lambda h u_{n+1}$ and therefore

$$u_{n+1} = \left(\frac{1}{1-\lambda h}\right)^{n+1}, n \geq 0 \tag{7.18}$$

which tends to zero as $n \to \infty$ for all values if $h > 0$. Similarly, with the Crank-Nicolson method we obtain

$$u_{n+1} = \left[\left(1 + \frac{h\lambda}{2}\right) / \left(1 - \frac{h\lambda}{2}\right)\right]^{n+1}, n \geq 0 \tag{7.19}$$

which still tends to zero as $n \to \infty$ for all values of $h > 0$. We can conclude that the forward Euler method is *conditionally absolutely stable*, while both the backward Euler and Crank-Nicolson methods are *unconditionally absolutely stable.*

8 Sparse matrices

8.1 Sparse matrices storage formats

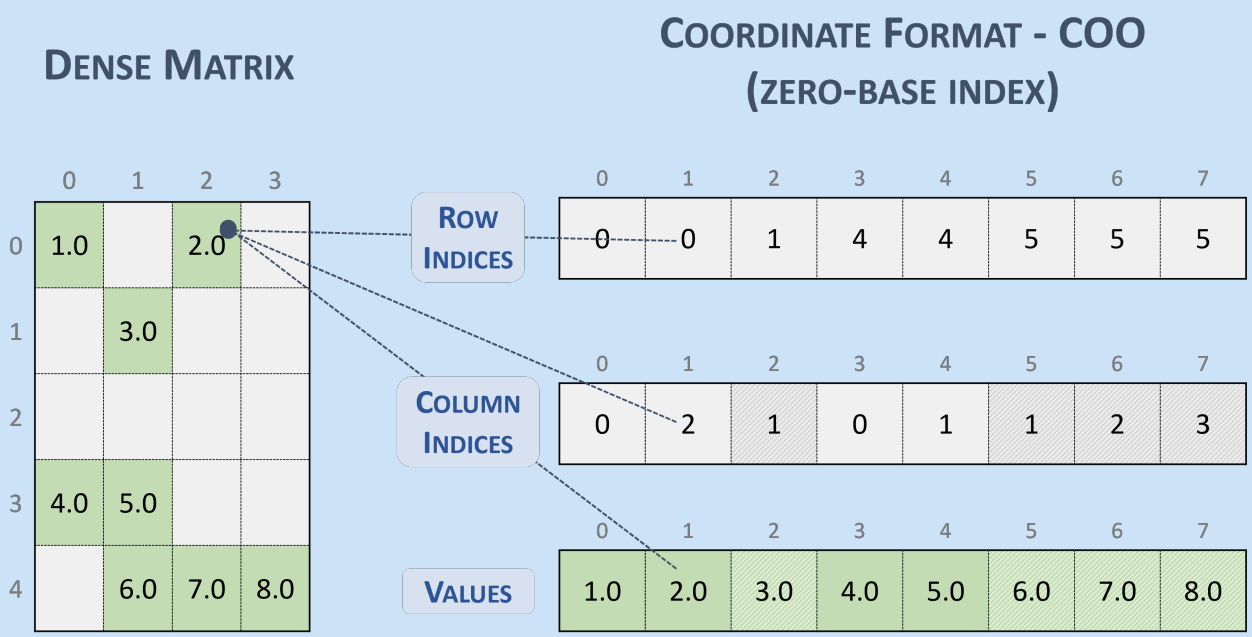
Sparse matrices are matrices that contain a large number of zero elements. The storage of these matrices can be optimized by using different formats. The most common formats are:

8.1.1 Coordinate format (COO)

The simplest storage scheme for sparse matrices is the so-called coordinate format. The data structure consists of three arrays:

1. `AA` - all the values of the nonzero elements of A in any order.
2. `JR` - the row indices of the nonzero elements of A.
3. `JC` - the column indices of the nonzero elements of A.

e.g. Coordinate format **example 8.1.1**



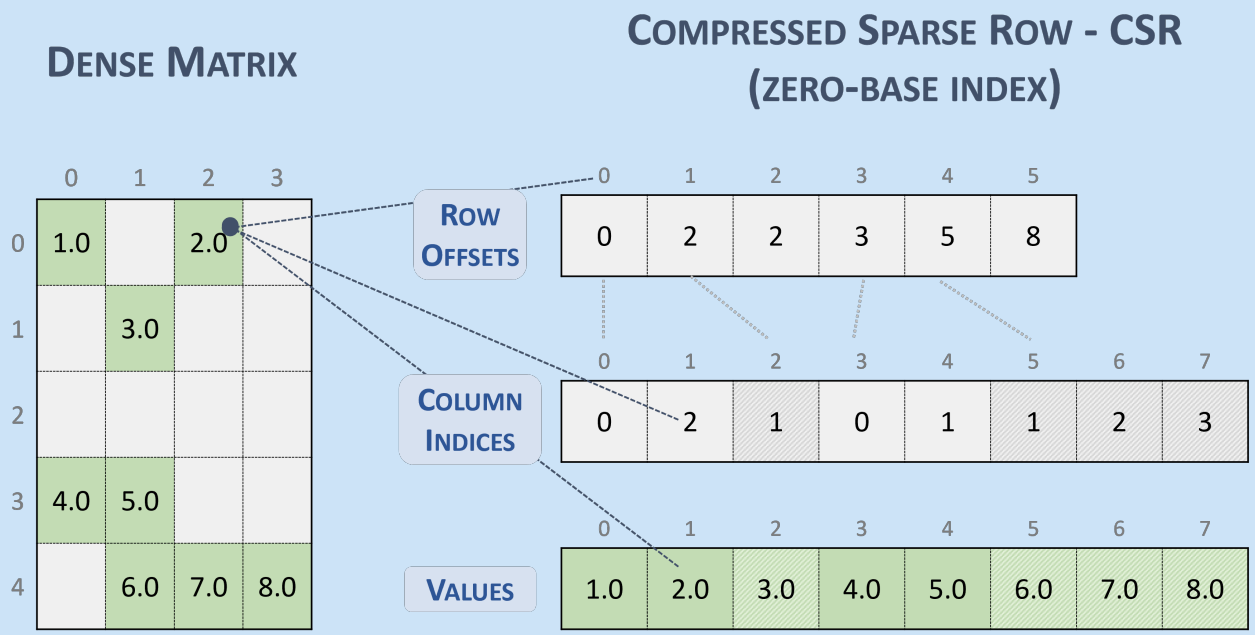
8.1.2 Compressed sparse row (CSR)

The CSR format is similar to COO, where the row indices are compressed and replaced by an array of offsets. The new data structure consists of three arrays:

1. `AA` - the real values a_{ij} sorted row by row, from row 1 to row n.
2. `JA` - the column indices of the nonzero elements of A.
3. `IA` - the row offsets. contains the pointers to the beginning of each row in the array A and JA. The content of IA is the position in the arrays AA and JA where the row i starts. The

length of IA is $n+1$, with $IA(n+1)$ containing the total number of nonzero elements in the matrix.

e.g. Compressed sparse row format **example 8.1.2**



To create a sparse matrix in the CSR format, we use the `csr_matrix` function, which is provided by the `scipy.sparse` module. Here is an example program:

```python
import scipy.sparse as sp
from scipy import *

data = [1.0, 2.0, -1.0, 6.6, 1.4]
rows = [0, 1, 1, 3, 3]
cols = [1, 1, 2, 0, 4]

A = sp.csr_matrix((data, [rows, cols]), shape=(4, 5))
print(A)

>>> A.data
array([ 1. ,  2. , -1. ,  6.6,  1.4])
>>> A.indices
array([1, 1, 2, 0, 4], dtype=int32)
>>> A.indptr
array([0, 1, 3, 3, 5], dtype=int32)
```

9 Direct Methods for Solving Linear Systems

9.1 Solution of Triangular Systems

Consider the nonsingular 3×3 **lower triangular** system:

$$\begin{bmatrix} l_{11} & 0 & 0 \\ l_{21} & l_{22} & 0 \\ l_{31} & l_{32} & l_{33} \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ x_3 \end{bmatrix} = \begin{bmatrix} b_1 \\ b_2 \\ b_3 \end{bmatrix} \tag{9.1}$$

Since the matrix is nonsingular, its diagonal entries l_{ii} are nonzero, hence we can solve sequentially for the unknown values x_i, as follows:

$$\begin{aligned} x_1 &= b_1/l_{11} \\ x_2 &= (b_2 - l_{21}x_1)/l_{22} \\ x_3 &= (b_3 - l_{31}x_1 - l_{32}x_2)/l_{33} \end{aligned} \tag{9.2}$$

This algorithm can be extended to systems $n \times n$ and is called ***forward substitution***. In the case of system $L\boldsymbol{x} = \boldsymbol{b}$, with L being a nonsingular lower triangular matrix of order $n(n \geq 2)$, the method is as follows:

$$\begin{aligned} x_1 &= b_1/l_{11} \\ x_i &= \frac{1}{l_{ii}} \left(b_i - \sum_{j=1}^{i-1} l_{ij}x_j \right), i = 2, ..., n \end{aligned} \tag{9.3}$$

The number of multiplications and divisions to execute the algorithm is equal to $\frac{n(n+1)}{2}$, while the number of sums and subtractions is $\frac{n(n-1)}{2}$. **The global operation count for the forward substitution is n^2.**

Similar conclusions can be drawn for a linear system $U\boldsymbol{x} = \boldsymbol{b}$, with U being a nonsingular upper triangular matrix of order $n(n \geq 2)$. In this case the algorithm is called ***backward substitution*** and in the general case can be written as:

$$\begin{aligned} x_n &= \frac{b_n}{u_{nn}} \\ x_i &= \frac{1}{u_{ii}} \left(b_i - \sum_{j=i+1}^{n} u_{ij}x_j \right), i = n-1, ..., 1 \end{aligned} \tag{9.4}$$

Its computational cost is the same as that of the forward substitution.

9.2 Guassian Elimination and LU Factorization

Consider a nonsingular matrix $A \in \mathbb{R}^{n \times n}$, and suppose that the diagonal entries a_{ii} is nonzero. Introducing the *multiplers*:

$$m_{i1} = \frac{a_{i1}^{(1)}}{a_{11}^{(1)}}, i = 2, ..., n \tag{9.5}$$

where $a_{i1}^{(1)}$ denote the elements of $A^{(1)}$, it is possible to eliminate the unknown x_1 from the rows other than the first one by simply subtracting from row i, with $i = 2, ..., n$, the first row multiplied by m_{i1} and doing the same on the right side. If we now define

$$\begin{aligned} a_{ij}^{(2)} &= a_{ij}^{(1)} - m_{i1} a_{1j}^{(1)}, i, j = 2, ..., n \\ b_i^{(2)} &= b_i^{(1)} - m_{i1} b_1^{(1)}, i = 2, ..., n \end{aligned} \tag{9.6}$$

where $b_i^{(1)}$ denotes the elements of $\boldsymbol{b}^{(1)}$, we have the following system:

$$\begin{bmatrix} a_{11}^{(1)} & a_{12}^{(1)} & ... & a_{1n}^{(1)} \\ 0 & a_{22}^{(2)} & ... & a_{2n}^{(2)} \\ ... & & & \\ 0 & 0 & ... & a_{nn}^{(n)} \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ ... \\ x_n \end{bmatrix} = \begin{bmatrix} b_1^{(1)} \\ b_2^{(2)} \\ ... \\ b_n^{(n)} \end{bmatrix} \tag{9.7}$$

which we denote by $A^{(2)}\boldsymbol{x} = \boldsymbol{b}^{(2)}$. that is equivalent to the starting one. Similarly, we can transform the system in such a way that the unknown x_2 is eliminated from rows $3, ..., n$,.In general, we end up with the finite sequence of systems

$$A^{(k)}\boldsymbol{x} = \boldsymbol{b}^{(k)}, k = 1, ..., n \tag{9.8}$$

where, for $k \geq 2$, matrix $A^{(k)}$ takes the following form:

$$A^{(k)} = \begin{bmatrix} a_{11}^{(1)} & a_{12}^{(1)} & ... & a_{1n}^{(1)} \\ 0 & a_{22}^{(2)} & ... & a_{2n}^{(2)} \\ ... & & & \\ 0 & 0 & ... & a_{kk}^{(k)} \end{bmatrix} = L^{(k)}U^{(k)} \tag{9.9}$$

Def elmentary lower triangular matrix **definition 9.2.1**

For $1 \leq k \leq n-1$, let $\boldsymbol{m} \in \mathbb{R}^n$ be a vector with $\boldsymbol{e}_j^T\boldsymbol{m} = 0$ for $1 \leq j \leq k$, meaning that $\boldsymbol{m}$ is of the form

$$\boldsymbol{m} = [0 \ ... \ 0 \ m_{k+1} \ ... \ m_n]^T \tag{9.10}$$

An elementary lower triangular matrix is a lower triangular matrix of the specific form:

$$L_{k(\boldsymbol{m})} := I - \boldsymbol{m}\boldsymbol{e}_k^T = \begin{bmatrix} 1 & & & & \\ & \ddots & & & \\ & & 1 & & \\ & & -m_{k+1} & 1 & \\ & & \vdots & & \ddots \\ & & -m_n & & 1 \end{bmatrix} \tag{9.11}$$

It is clear that for $k = n$ we obtain the upper triangular system $U\boldsymbol{x} = \boldsymbol{b}^{(n)}$ which can be solved by backward substitution.

GAUSSIAN ELIMINATION

Input: $A \in \mathbb{R}^{n\times n}$, $\boldsymbol{b} \in \mathbb{R}^n$

Output: $A^{(n)}$upper triangular and $\boldsymbol{b}^{(n)}$

For $k = 1$ to $n-1$ **do**

  $d := 1/a_{kk}$

  For $i = k+1$ to n **do**

    $a_{ik} := a_{ik} \cdot d$

    $b_i = b_i - b_k \cdot a_{ik}$

    For $j = k+1$ to n **do**

      $a_{ij} = a_{ij} - a_{ik}a_{kj}$

Gaussian elimination requires $O(n^3)$ time and $O(n^2)$ space.

Suppose we may construct a sequence of $n-1$ elementary lower triangular matrices $L_j = L_j(\boldsymbol{m_j})$, such that

$$L_{n-1}L_{n-2}...L_2L_1A = U \tag{9.12}$$

To sovle the system $A\boldsymbol{x} = \boldsymbol{b}$, we can use the following algorithm:

$$\begin{cases} A\boldsymbol{x} = \boldsymbol{b} \\ A = LU \end{cases} \longrightarrow LU\boldsymbol{x} = \boldsymbol{b} \longrightarrow \begin{cases} L\boldsymbol{y} = \boldsymbol{b} \\ U\boldsymbol{x} = \boldsymbol{y} \end{cases} \tag{9.13}$$

Thm Existence and Uniqueness **theorem 9.2.1**

Let $A \in \mathbb{R}^{n\times n}$. The LU factorization of A with $l_{ii} = 1$ for $i = 1, ..., n$ exists and is unique iff the principal submatrices A_i of A of order $i = 1, ..., n-1$ are nonsingular.

Thm Sufficient Condition for Gaussian Elimination **theorem 9.2.2**

Let $A \in \mathbb{R}^{n\times n}$ be a nonsingular matrix. The LU factorization of A exists and is unique if A follows the below two conditions:

1. A is strictly diagonally dominant by rows / columns. $|a_{ii}| \geq \sum_{j=1, j\neq i}^{n} |a_{ij}|$
2. A is symmetric and positive definite matrix. $A = A^T$ and $\boldsymbol{x}^T A\boldsymbol{x} > 0$.

9.3 Pivoting techniques

As previously pointed out, the GEM process breaks down as soon as a zero pivotal entry is computed. In such case, one needs to resort to the so-called *pivoting techniques*, which amounts to exchanging rows(columns) of the system in such a way that nonzero pivotal elements are always available. So the LU factorization becomes:

$$PA = LU \tag{9.14}$$

where P is a permutation matrix. To solve linear system $A\boldsymbol{x} = \boldsymbol{b}$, we solve the equivalent system $PA\boldsymbol{x} = P\boldsymbol{b}$, which can be solved by the following two triangular systems:

$$\begin{cases} A\boldsymbol{x} = \boldsymbol{b} \\ PA = LU \end{cases} \longrightarrow PA\boldsymbol{x} = P\boldsymbol{b} \longrightarrow \begin{cases} L\boldsymbol{y} = P\boldsymbol{b} \\ U\boldsymbol{x} = \boldsymbol{y} \end{cases} \tag{9.15}$$

Moreover, the piovtal element should be as large as possible to avoid round-off errors. In practice:

1. doing pivoting even when it is not strictly needed.
2. Swap the row k with the row i, where i is the row with the largest pivotal element in the k -th column.

LU Factorization with partial pivoting

Input: $A \in \mathbb{R}^{n\times n}$, $\boldsymbol{b} \in \mathbb{R}^n$

Output: $PA = LU$

For $k = 1$ to $n-1$ **do**

  Find the pivot element a_{kr} for row k

  Exchange row k with row r

  $d := 1/a_{kk}$

  For $i = k+1$ to n **do**

    $a_{ik} := a_{ik} \cdot d$

    $b_i = b_i - b_k \cdot a_{ik}$

    For $j = k+1$ to n **do**

      $a_{ij} = a_{ij} - a_{ik}a_{kj}$

9.4 Cholesky Factorization

Let us assume that A has an LU factorization $A = LU$ and that A is symmetric, $A = A^T$. Then we have:

$$A = LU = A^T = U^T L^T \tag{9.16}$$

Since U^T is a lower triangular matrix and L^T is an upper triangular matrix, we would like to use the uniqueness of the LU factorization to conclude that $L = U^T, U = L^T$. Unfortunately, this is not possible since the uniqueness requires the lower triangular matrix to be normalised, i.e. to have only ones as diagonal entries, and this may not be the case for U^T.

But if A is invertible, then we can write

$$U = D\tilde{U} \tag{9.17}$$

with a diagonal matrix $D = \mathrm{diag}(d_{11}, ..., d_{nn})$ and a normalized upper triangular natrix $\tilde{U}$. Therefore, we can conclude that $A = LD\tilde{U}$ and hence $A^T = \tilde{U}^T DL^T = LD\tilde{U}$, so that we can now apply the uniqueness result to derive $L = \tilde{U}^T$ and $U = DL^T$, which gives $A = LDL^T$.

Let us finally make the assumption that all diagonal entries of D are positive, so that we can define its square root by setting $D^{\frac{1}{2}} = \text{diag}\left(\sqrt{d_{11}}, ..., \sqrt{d_{nn}}\right)$ which leads to the decomposition

$$A = LD^{\frac{1}{2}}D^{\frac{1}{2}}L^T = \tilde{L}\tilde{L}^T \tag{9.18}$$

Def Cholesky Factorization **definition 9.4.1**

A decomposition of a matrix A of the form $A = LL^T$ with a lower triangular matrix L is called the ***Cholesky factorization*** of A.

It remains to verify for what kind of matrices a Cholesky factorisation exists.

Thm Cholesky factorization **theorem 9.4.1**

Suppose $A = A^T$ is positive define. Then, A possesses a Cholesky factorization.

CHOLESKY FACTORIZATION

Let $r_{11} = \sqrt{a_{11}}$.

For $k = 2, ...$

$$r_{ij} = \frac{1}{r_{ii}}\left(a_{ij} - \sum_{k=1}^{i-1} r_{ki}r_{kj}\right)$$
$$r_{ii} = \sqrt{a_{ii} - \sum_{k=1}^{i-1} r_{ki}^2}$$

The computational complexity of computing the Cholesky factorization is given by $n^3/6 + O(n^2)$, which is about half the complexity of the standard Gaussian elimination process.

9.5 Fill-in

When A is sparse, LU decomposition results in L, U with nonzeros(called ***fill-in***) at positions that were originally zero.

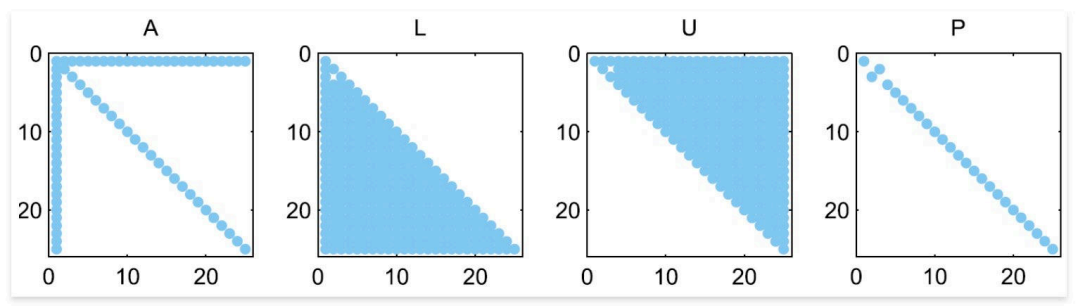


Figure 9.6: Fill-in in the LU decomposition of a sparse matrix

Gaussian Elimination (in its plain version) is made in such a way as to memorise L and U by overwriting the space allocated for A. So LU factorization is not suitable (from the point of view of memory occupation) for sparse matrices.

To reduce fill-in, it may be useful to employ reordering techniques, which consists in numbering the rows of A differently. The aim is to reduce the number of nonzeros in L, U by permuting the nonzero structure of A into a special form and respecting this form when performing the reordering.

10 Preconditioning

11 Iterative methods for large linear systems

Given an $n \times n$ real matrix A and a real n-vector, the problem is: Find $\boldsymbol{x}$ belonging to $\mathbb{R}^n$ such that

$$A\boldsymbol{x} = \boldsymbol{b} \tag{11.1}$$

where $\boldsymbol{x}$ is the exact solution of the linear system $A\boldsymbol{x} = \boldsymbol{b}$. In such cases existence and uniqueness of the solution are ensured if one of the following (equivalent) hypotheses holds:

1. A is invertible
2. rank(A)=n;
3. the homogeneous system $A\boldsymbol{x} = 0$ admits only the null solution.

In general, a direct method for solving the linear system requires $O(n^3)$ time. The basic idea behind iterative methods is to produce a way of approximating the application of the inverse of A to $\boldsymbol{b}$. The computation of each iteration usually costs $O(n^2)$ time so that the method becomes computationally interesting only if a sufficiently good approximation can be achieved in far fewer than n steps.

11.1 Banach Fixed Point Theorem

We will start by deriving a general convergence theory for an iteration process. even with a more general, not necessarily linear $F : \mathbb{R}^n \to \mathbb{R}^n$.

Def contraction mapping **definition 11.1.1**

A mapping $F : \mathbb{R}^n \to \mathbb{R}^n$ is called a *contraction mapping* with repect to a norm $\|\cdot\|$ on $\mathbb{R}^n$ if there exists a constant $0 \leq q \leq 1$ such that

$$\|\boldsymbol{F}(\boldsymbol{x}) - \boldsymbol{F}(\boldsymbol{y})\| \leq q \, \|\boldsymbol{x} - \boldsymbol{y}\|, \forall x, y \in \mathbb{R}^n \tag{11.2}$$

A contraction mapping is Lipschitz continuous with Lipschitz constant $q < 1$.

Thm Banach Fixed Point Theorem **theorem 11.1.1**

Let $F : \mathbb{R}^n \to \mathbb{R}^n$ be a contraction mapping with respect to a norm $\|\cdot\|$ on $\mathbb{R}^n$. Then, there exists a unique fixed point $x^* \in \mathbb{R}^n$ such that $F(x^*) = x^*$. The sequence $\boldsymbol{x}_{j+1} := F(\boldsymbol{x}_j)$ converges for every starting point $\boldsymbol{x}_0 \in \mathbb{R}^n$ to the fixed point x^*. Furthermore, we have the error estimates

$$\begin{aligned} \|\boldsymbol{x}^* - \boldsymbol{x_j}\| &\leq \frac{q^j}{1-q} \, \|\boldsymbol{x_1} - \boldsymbol{x_0}\| \quad \text{priori} \\ \|\boldsymbol{x}^* - \boldsymbol{x_j}\| &\leq \frac{q}{1-q} \, \|\boldsymbol{x_j} - \boldsymbol{x_{j-1}}\| \quad \text{posteriori} \end{aligned} \tag{11.3}$$

11.2 On the Convergence of Iterative Methods

The basic idea of iterative methods is to construct a sequence of vectors $\boldsymbol{x^k}$ that enjoy the property of *convergence*

$$\boldsymbol{x} = \lim_{k\to\infty} \boldsymbol{x^k} \tag{11.4}$$

In practice, the iterative process is stopped at the minimum value of n such that $\|\boldsymbol{x}^{(\boldsymbol{n})} - \boldsymbol{x}\| < \varepsilon$, where ε is a given tolerance and $\|\cdot\|$ is a suitable norm. However, since the exact solution is obviously not available, it is necessary to introduce suitable stopping criteria to monitor the convergence of the iteration.

To start with, we consider iterative methods of the form

$$\text{Given } \boldsymbol{x}^0, \boldsymbol{x}^{k+1} = B\boldsymbol{x^k} + \boldsymbol{f}, k \geq 0 \tag{11.5}$$

where B is an $n \times n$ square matrix called the *iteration matrix* and $\boldsymbol{f}$ is a vector that is obtained from the right-hand side $\boldsymbol{b}$.

having denoted by B an $n \times n$ square matrix called the iteration matrix and by $\boldsymbol{f}$ a vector that is obtained from the right hand side $\boldsymbol{b}$.

Def Consistent **definition 11.2.1**

An iterative method of the form Eq. (11.5) is said to be *consistent* with Eq. (11.1) if $\boldsymbol{f}$ and B are such that $\boldsymbol{x} = \boldsymbol{Bx} + \boldsymbol{f}$. Equivalently,

$$\boldsymbol{f} = (1 - B)A^{-1}\boldsymbol{b} \tag{11.6}$$

Having denoted by

$$\boldsymbol{e}^{(k)} = \boldsymbol{x}^{(k)} - \boldsymbol{x} \tag{11.7}$$

the error at the k-th step of the iteration, the condition for convergence amounts to requiring that $\lim_{k\to\infty}\|\boldsymbol{e^k}\| = 0$ for any choice of the initial datum $\boldsymbol{x}^0$.

Consistency alone does not suffice to ensure the convergence of the iterative method Eq. (11.5).

Thm Convergence of Iterative method **theorem 11.2.1**

Let Eq. (11.5) be a consistent method. Then, the sequence of vectors $\{x^k\}$ converges to the solution of Eq. (11.1) for any choice of $x^{(0)}$ iff $\rho(B) < 1$.

Proof. From Eq. (11.7) and the consistency assumption, the recursive relation $\boldsymbol{e}^{k+1} = Be^k$ is obtained:

$$\boldsymbol{e}^{k+1} = x^{k+1} - x^k = Bx^k + f - (Bx + f) = Be^k \tag{11.8}$$

Therefore,

$$e^{(k)} = B^k e^{(0)}, \forall k = 0, 1, ... \tag{11.9}$$

Thus, thanks to theorem 2.3.1, it follows that $\lim_{k\to\infty} B^k e^0 = 0$ for any $e^{(0)}$ iff $\rho(B) < 1$.

Def **definition 11.2.2**

Let B be the iteration matrix. We call:

1. $\|\boldsymbol{B^m}\|$ the *convergence factor* after m steps of the iteration.
2. $\|\boldsymbol{B}\|^{1/m}$ the *average convergence* factor after m steps;
3. $R_{m(B)} = -\frac{1}{m}\log\|\boldsymbol{B^m}\|$ the *average convergence rate* after m steps.

11.3 The Jacobi and Gauss-Seidel Methods

After this general discussion, we return to the question of how to choose the iteration matrix B. Our initial approach is based upon a splitting

$$B = C^{-1}(C - A) = I - C^{-1}A, \quad \boldsymbol{b} = C^{-1}\boldsymbol{b} \tag{11.10}$$

with a matrix C, which should be sufficiently close to A but also easily invertible.

Next, we decompose A in its lower-left sub-diagonal part, its diagonal part and its upper-right super-diagonal part

$$A = L + D + R \tag{11.11}$$

with

$$L = \begin{cases} a_{ij} \text{ if } i > j \\ 0 \text{ else} \end{cases} \quad D = \begin{cases} a_{ij} \text{ if } i = j \\ 0 \text{ else} \end{cases} \quad R = \begin{cases} a_{ij} \text{ if } i < j \\ 0 \text{ else} \end{cases} \tag{11.12}$$

The simplest possible approximation to A is then given by picking its diagonal part D for B so that the iteration matrix becomes

$$B_J = I - C^{-1}A = I - D^{-1}(L + D + R) = -D^{-1}(L + R) \tag{11.13}$$

with entries

$$b_{ij} = \begin{cases} -a_{ij}/a_{ii} \text{ if } i \neq j \\ 0 \text{ else} \end{cases} \tag{11.14}$$

This means that we can write the iteration defined by

$$\boldsymbol{x}^{k+1} = -D^{-1}(L + R)\boldsymbol{x}^k + D^{-1}\boldsymbol{b} \tag{11.15}$$

In the Jacobi method, once an arbitrarily initial guess $x^{(0)}$ is given, the solution is updated by the formula:

$$x_i^{(k+1)} = \frac{1}{a_{ii}}\left(b_i - \sum_{\substack{j=1 \\ j\neq i}}^{n} a_{ij}x_j^{(k)}\right), i = 1, ..., n \tag{11.16}$$

In general, each iteration costs $O(n^2)$ operations, so the Jacobi method is competitive if the number of iterations is less than n. If A is sparse matrix, then the cost is only n flops per iteration.

It should be noted that the solutions $\boldsymbol{x}_i^{(k+1)}$ can be computed fully in parallel (very competitive for large scale systems).

Thm Convergence of the Jacobi Method **theorem 11.3.1**

The Jacobi method converges for every starting point if the matrix A is strictly row diagonally dominant.

Proof: We use the infinity norm to calculate the norm of the iteration matrix B_J as

$$\|\boldsymbol{B_J}\| = \max_{1\leq i\leq n}\sum_{k=1}^{n}|b_{ik}| = \max_{1\leq i\leq n}\sum_{k=1,k\neq i}^{n}\frac{|a_{ik}|}{|a_{ii}|} \tag{11.17}$$

The Gauss-Seidel method differs from the Jacobi method in the fact that at the $k+1$th step the available values of $x_i^{(k+1)}$ are being used to update the solution:

$$x_i^{(k+1)} = \frac{b_i - \sum_{j=1}^{i-1} a_{ij}x_j^{(k+1)} - \sum_{j=i+1}^{n} a_{ij}x_j^{(k)}}{a_{ii}}, i = 1, ..., n \tag{11.18}$$

To see that this scheme is consistent and to analyse its convergence, we have to find iteration matrix B. We rewrite Eq. (11.18) as

$$a_{ii}x_i^{(k+1)} + \sum_{j=1}^{i-1} a_{ij}x_j^{(k+1)} = b_i - \sum_{j=i+1}^{n} a_{ij}x_j^{(k)} \tag{11.19}$$

which translates into $(L+D)\boldsymbol{x}^{(k+1)} = -R\boldsymbol{x}^{(k)} + b$. Hence, if we define $C = (L+D)^{-1}$ and use $C - A = -R$, we note that the scheme is indeed consistent and that the iteration matrix of the Gauss-Seidel method is given by

$$B_{GS} = -(L+D)^{-1}R \tag{11.20}$$

Thm Convergence of the Gauss-Seidel Method **theorem 11.3.2**

If $A \in \mathbb{R}^{n\times n}$ is symmetrice and positive definite then the Gauss-Seidel method converges.

If A is tridiagonal, Jacobi will converge iff the Gauss-Seidel method also converges.

11.4 Stopping Criteria

The convergence of an iterative method is monitored by means of a stopping criterion. We can easily introduce the following criteria:

$$\frac{\|\boldsymbol{x} - \boldsymbol{x}^{(k)}\|}{\|\boldsymbol{x}^{(k)}\|} \leq \varepsilon \tag{11.21}$$

Unfortunately, the exact solution $\boldsymbol{x}$ is not known, we are trying to convert the problem into a residual-based stopping criterion.

Residual based stopping criteria: The iteration is stopped when the residual $\boldsymbol{r}^{(k)} = \boldsymbol{b} - A\boldsymbol{x}^{(k)}$ is small enough:

$$\frac{\|\boldsymbol{x} - \boldsymbol{x}^{(k)}\|}{\|\boldsymbol{x}^{(k)}\|} \leq K(A)\frac{\|\boldsymbol{r}^{(k)}\|}{\|\boldsymbol{b}\|} \Longrightarrow \frac{\|\boldsymbol{r}^{(k)}\|}{\|\boldsymbol{b}\|} \leq \varepsilon \tag{11.22}$$

This is a good criteria whenever the condition number $K(A)$ is not too large. If the condition number is large, the residual-based stopping criterion may be too stringent. To make the constant $K(A)$ smaller in the stopping criterion, there is a method called *preconditioning*:

$$\frac{\|\boldsymbol{x} - \boldsymbol{x}^{(\boldsymbol{k})}\|}{\|\boldsymbol{x}^{(\boldsymbol{k})}\|} \leq K(P^{-1}A)\frac{\|\boldsymbol{z}^{(\boldsymbol{k})}\|}{\|\boldsymbol{b}\|} \Longrightarrow \frac{\|\boldsymbol{z}^{(\boldsymbol{k})}\|}{\|\boldsymbol{b}\|} \leq \varepsilon \tag{11.23}$$

where $z^{(k)} = P^{-1}\boldsymbol{r}^k$.

Distance between consecutive iterations: The iteration is stopped when the distance between consecutive iterates is small enough, define the distance $\boldsymbol{\delta}^{(k)} = \boldsymbol{x}^{(k+1)} - \boldsymbol{x}^{(k)}$, then the stopping criterion is:

$$\|\boldsymbol{\delta}^{(\boldsymbol{k})}\| \leq \varepsilon \tag{11.24}$$

The relation between the true error and the distance between consecutive iterates is given by:

$$\|\boldsymbol{e}^{(\boldsymbol{k})}\| \leq \frac{\|\boldsymbol{\delta}^{(\boldsymbol{k})}\|}{1 - \rho(B)} \tag{11.25}$$

Therefore this is a "good" stopping criterion only if $\rho(B) \ll 1$.

A general technique to devise consistent linear iterative methods is based on an additive splitting of the matrix A of the form $A = P - N$, where P and N are two suitable matrices and P is nonsingular. For reasons that will be clear in the later sections, P is called *preconditioning matrix or preconditioner*.

Precisely, given $x^{(0)}$, one can compute $x^{(k)}$ for $k \geqslant$, solving the system:

$$Px^{(k+1)} = Nx^{(k)} + b \tag{11.26}$$

The iteration matrix of method Eq. (11.26) is $B = P^{-1}N$ and the vector $f = P^{-1}b$. Alternatively, the method can be written as:

$$x^{(k+1)} = x^{(k)} + P^{-1}r^{(k)} \tag{11.27}$$

where the residual

$$r^{(k)} = b - Ax^{(k)} \tag{11.28}$$

is the vector that measures the error in the approximation $x^{(k)}$. Eq. (11.27) outlines the fact that a linear system, with coefficient matrix P, must be solved to update the solution at step $k+1$. Thus P, besides being nonsingular, ought to be easily invertible, in order to keep the overall computational cost low.(Notice that, if P were equal to A and $N = 0$, method Eq. (11.27) would converge in one iteration, but at the same cost of a direct method.

Let us mention two results that ensure convergence of the iteration Eq. (11.27), provided suitable conditions on the splitting of A are fulfilled.

11.5 Richardson Iteration

Devoted by

$$R_p = I - P^{-1}A \tag{11.29}$$

the iteration matrix associated with Eq. (11.27). Proceeding as in the case of relaxation methods, Eq. (11.27) can be generalized introducing a relaxation (or acceleration) parameter α. This leads to the following *stationary Richardson method.*

$$\boldsymbol{x}^{(k+1)} = \boldsymbol{x}^{(k)} + \alpha P^{-1}\boldsymbol{r}^{(k)}, k \geq 0 \tag{11.30}$$

More generally, allowing α to depend on the iteration index, *the nonstationary Richardson method or semi-iterative method* is given by

$$\boldsymbol{x}^{(k+1)} = \boldsymbol{x}^{(k)} + \alpha^{(k)} P^{-1}\boldsymbol{r}^{(k)}, k \geq 0 \tag{11.31}$$

The iteration matrix at the k-th step for Eq. (11.31) is

$$B_R = I - \alpha_k P^{-1} A \tag{11.32}$$

with $\alpha_k = \alpha$ in the stationary case. If $P = I$, the family of methods Eq. (11.31) will be called *nonpreconditioned.*The Jacobi and Gauss-Seidel methods can be regarded as stationary Richardson methods with $P = D$ and $P = D - E$, respectively.

We can rewrite Eq. (11.31) in a form of greater interest for computation. Letting $\boldsymbol{z}^{(k)} = P^{-1}\boldsymbol{r}^{(k)}$(the so-called *preconditioned residual*), we have $\boldsymbol{x}^{(k+1)} = \boldsymbol{x}^{(k)} + \alpha_k \boldsymbol{z}^{(k)}$ and $\boldsymbol{r}^{(k+1)} = \boldsymbol{r}^{(k)} - \alpha_k A\boldsymbol{z}^{(k)}$.

To summarize, a nonstationary Richardson method requires at each $k+1$th step the following operations:

1. solve the linear system $P\boldsymbol{z}^{(k)} = \boldsymbol{r}^{(k)}$
2. compute the acceleration parameter α_k
3. update the solution $\boldsymbol{x}^{(k+1)} = \boldsymbol{x}^{(k)} + \alpha_k \boldsymbol{z}^{(k)}$
4. update the residual $\boldsymbol{r}^{(k+1)} = \boldsymbol{r}^{(k)} - \alpha_k A\boldsymbol{z}^{(k)}$

Thm Convergence of stational Richardson method **theorem 11.5.1**

Assume that P is a nonsingular matrix and that $P^{-1}A$ has positive real eigenvalues, ordered in such a way that $\lambda_1 \geq \lambda_2 \geq ... \geq \lambda_n > 0$. Then, the stationary Richardson method converges if and only if $0 \leq \alpha \leq \frac{2}{\lambda_1}$. Moreover, letting

$$\alpha_{\text{opt}} = \frac{2}{\lambda_1 + \lambda_n} \tag{11.33}$$

the spectral radius of the iteration matrix R_α is minimum if $\alpha = \alpha_{\text{opt}}$, with

$$\rho_{\text{opt}} = \min_{\{\alpha\}} \rho(R_\alpha) = \frac{\lambda_1 - \lambda_n}{\lambda_1 + \lambda_n} \tag{11.34}$$

Thm Convergence of nonstationary Richardson method **theorem 11.5.2**

Under the same assumption on P and A, the dynamic Richardson method converges if for instance α_k is chosen in the following way:

$$\alpha_k = \frac{\left(\boldsymbol{z}^{(k)}\right)^T \boldsymbol{r}^{(k)}}{\left(\boldsymbol{z}^{(k)}\right)^T A\boldsymbol{z}^{(k)}} \tag{11.35}$$

where $\boldsymbol{z}^{(k)} = P^{-1}\boldsymbol{r}^{(k)}$ is the preconditioned residual.

For both choices, Eq. (11.33) and Eq. (11.35), the following error estimates hold:

$$\left\|\boldsymbol{e}^{(k)}\right\|_A \leq \left(\frac{K(P^{-1}A)-1}{K(P^{-1}A)+1}\right)^k \left\|\boldsymbol{e}^{0}\right\|_A \tag{11.36}$$

e.g. Preconditioned Stational Richardson Method **example 11.5.1**

PRECONDITIONED STATIONAL RICHARDSON METHOD

Given arbitrary $\boldsymbol{x}^0 =$, Let $\boldsymbol{r}^0 = \boldsymbol{b} - A\boldsymbol{x}^0$

For $k = 0, 1, ...$

> Compute $\alpha_{\text{opt}} = \frac{2}{\lambda_{\min}(P^{-1}A)+\lambda_{\max}(P^{-1}A)}$
> Solve the linear system $P\boldsymbol{z}^{(k)} = \boldsymbol{r}^{(k)}$
> Update the solution: $\boldsymbol{x}^{(k+1)} = \boldsymbol{x}^{(k)} + \alpha_{\text{opt}}\boldsymbol{z}^{(k)}$
> Update the residual: $\boldsymbol{r}^{(k+1)} = \boldsymbol{r}^{(k)} - \alpha_{\text{opt}}A\boldsymbol{z}^{(k)}$

11.5.1 Preconditioning Matrices

All the methods introduced in the previous sections can be cast in the form Eq. (11.5) , so that they can be regarded as being methods for solving the system

$$(I - B)\boldsymbol{x} = \boldsymbol{f} = P^{-1}\boldsymbol{b} \tag{11.37}$$

On the other hand, since $B = P^{-1}N$, linear system $A\boldsymbol{x} = \boldsymbol{b}$ can be equivalently rewrited as

$$P^{-1}A\boldsymbol{x} = P^{-1}\boldsymbol{b} \tag{11.38}$$

The latter is the preconditioned system, being P the preconditioning matrix or left preconditioner. Right and centered preconditioners can be introduced as well,

$$AP^{-1}\boldsymbol{y} = \boldsymbol{b}, \boldsymbol{y} = P\boldsymbol{x} \tag{11.39}$$

or

$$P_L^{-1}AP_R^{-1}\boldsymbol{y} = P_L^{-1}\boldsymbol{b}, \boldsymbol{y} = P_R\boldsymbol{x} \tag{11.40}$$

There are *point preconditioners* and *block preconditioners*, depending on whether they are applied to the single entries of A or to the blocks of a partition of A. The iterative methods considered so far correspond to fixed-point iterations on a left-preconditioner system. Computing the inverse of P is not mandatory; actually, the role of P is to "precondition" the residual $\boldsymbol{r}^{(k)}$.

Since the preconditioner acts on the spectral radius of the iteration matrix, it would be useful to pick up, for a given linear system, an *optimal preconditioner*, a preconditioner which is able to make the number of iterations required for convergence independent of the size of the system.

There is not a general roadmap to devise optimal preconditioners. However, an established "rule of thumb" is that P is a good preconditioner for A if $P^{-1}A$ is near to being a normal matrix and if its eigenvalues are clustered within a sufficiently samll region of the complex field. The choice of a preconditioner must also be guided by practical considerations, noticeably, its computational cost and its memory requirements.

Preconditioners can be divided into two main categories: algebraic and functional preconditioners, the difference being that the algebraic preconditioners are independent of the problem that originated the system to be solved, and are actually constructed via algebraic procedures, while the functional preconditioners take advantage of the knowledge of the problem and are constructed as a function of it.

11.6 The Gradient Method

In the special case of symmetric and positive definite matrices, however, the optimal acceleration parameter can be dynamically computed at each step k as follows.

We first notice that, for such matrices, solving system Eq. (11.27) is equivalent to minimizing the quadratic form

$$\Phi(\boldsymbol{y}) = \frac{1}{2}\boldsymbol{y}^T A\boldsymbol{y} - \boldsymbol{y}^T b \tag{11.41}$$

which is called the *energy of system*, Indeed, the gradient of Φ is given by

$$\nabla\Phi(\boldsymbol{y}) = \frac{1}{2}(A + A^T)\boldsymbol{y} - \boldsymbol{b} = A\boldsymbol{y} - \boldsymbol{b} \tag{11.42}$$

As a consequence, if $\nabla\Phi(\boldsymbol{x}) = 0$, then $\boldsymbol{x}$ is a solution of the original system. Conversely, if $\boldsymbol{x}$ is a solution, then

$$\Phi(\boldsymbol{y}) = \Phi(\boldsymbol{x} + \boldsymbol{y} - \boldsymbol{x}) = \Phi(\boldsymbol{x}) + \frac{1}{2}(\boldsymbol{y} - \boldsymbol{x})^T A(\boldsymbol{y} - \boldsymbol{x}) \tag{11.43}$$

and thus, for any $\boldsymbol{y}$, the value of Φ is minimized at $\boldsymbol{x}$. Notice that the previous relation is equivalent to

$$\frac{1}{2}\|\boldsymbol{y} - \boldsymbol{x}\|_A^2 = \Phi(\boldsymbol{y}) - \Phi(\boldsymbol{x}) \tag{11.44}$$

where $\|\cdot\|_A$ is the energy norm, defined in theorem 2.1.4.

The problem is thus to determine the minimizer $\boldsymbol{x}$ of Φ by starting from a point $\boldsymbol{x}^0$, and, consequently, to select suitable directions along which moving to get as close as possible to the solution $\boldsymbol{x}$. The optimal direction, that joins the starting point $\boldsymbol{x}^{(0)}$ to the solution point $\boldsymbol{x}$, is obviously unknown a priori. Therefore, we must take a step from $\boldsymbol{x}^{(0)}$ along a given direction $\boldsymbol{p}_0$ and then fix along this latter a new point $\boldsymbol{x}^{(1)}$ from which to iterate the process until convergence.

Precisely, at the generic step k, $\boldsymbol{x}^{(k+1)}$ is computed as

$$\boldsymbol{x}^{(k+1)} = \boldsymbol{x}^{(k)} + \alpha_k \boldsymbol{p}^{(k)} \tag{11.45}$$

where α_k is the value which fixes the length of the step along the direction $\boldsymbol{p}^{(k)}$.

The most natural idea is to take as $\boldsymbol{p}^{(k)}$ the direction of maximum descent along the functional Φ in $\boldsymbol{x}^{(k)}$, which is given by $-\nabla\Phi\big(\boldsymbol{x}^{(k)}\big)$. This yields the gradient method, also called steepest descent method.

Due to Eq. (11.42), $\nabla\Phi\big(\boldsymbol{x}^{(k)}\big) = A\boldsymbol{x}^{(k)} - \boldsymbol{b} = -\boldsymbol{r}^{(k)}$ so that the direction of the gradient of Φ coincides with the residual $\boldsymbol{r}^{(k)}$. So it can be immediately computed using the current iterate. This shows that the gradient method, as well as the nonstationary Richardson method with $P = I$, moves at each step k along the direction of the residual $\boldsymbol{r}^{(k)}$.

To compute the parameter $\alpha^{(k)}$ let us write explicitly $\Phi\big(\boldsymbol{x}^{(k+1)}\big)$ as a function of a parameter α

$$\Phi\big(\boldsymbol{x}^{(k+1)}\big) = \frac{1}{2}\big(\boldsymbol{x}^{(k)} + \alpha\boldsymbol{r}^{(k)}\big)^T A\big(\boldsymbol{x}^{(k)} + \alpha\boldsymbol{r}^{(k)}\big) - \big(\boldsymbol{x}^{(k)} + \alpha\boldsymbol{r}^{(k)}\big)^T\boldsymbol{b} \tag{11.46}$$

Differentiating with respect to α and setting it equal to zero yields the desired value of α_k

$$\alpha_k = \frac{\boldsymbol{r}^{(k)^T}\boldsymbol{r}^{(k)}}{\boldsymbol{r}^{(k)^T}A\boldsymbol{r}^{(k)}} \tag{11.47}$$

GRADIENT METHOD

Given $\boldsymbol{x}^{(0)}$, compute the residual: $\boldsymbol{r}^{(0)} = \boldsymbol{b} - A\boldsymbol{x}^{(0)}$

While(Stopping criterion is not satisfied)

> Compute parameter $\alpha_k = \frac{\boldsymbol{r}^{(k)^T}\boldsymbol{r}^{(k)}}{\boldsymbol{r}^{(k)^T}A\boldsymbol{r}^{(k)}}$
>
> Update the solution: $\boldsymbol{x}^{(k+1)} = \boldsymbol{x}^{(k)} + \alpha_k\boldsymbol{r}^{(k)}$
>
> Update the residual: $\boldsymbol{r}^{(k+1)} = \boldsymbol{r}^{(k)} - \alpha_k A\boldsymbol{r}^{(k)}$ or $\big(\boldsymbol{b} - A\boldsymbol{x}^{(k+1)}\big)$

Thm Convergence of Gradient Method **theorem 11.6.1**

Let A be a symmetric and positive definite matrix. Then the gradient method is convergent for any choice of the initial datum $\boldsymbol{x}^{(0)}$. Moreover

$$\|\boldsymbol{e}^{(\boldsymbol{k+1})}\|_A \leq \frac{K(A)-1}{K(A)+1}\|\boldsymbol{e}^{(\boldsymbol{k})}\|_A = \left(\frac{K(A)-1}{K(A)+1}\right)^k \|\boldsymbol{e}^{(\boldsymbol{0})}\|_A \tag{11.48}$$

where $\|\cdot\|_A$ is the energy norm defined in theorem 2.1.4.

11.7 The Conjugate Gradient Method

The gradient method consists essentially of two phases: choosing a direction $\boldsymbol{p}^{(k)}$ and picking up a point of local minimum for Φ along that direction. The latter request can be accommodated by

choosing α_k as the value of the parameter α such that $\Phi\big(\boldsymbol{x}^{(k)} + \alpha \boldsymbol{p}^{(k)}\big)$ is minimized. Differentiating with respect to α and setting it equal to zero, we obtain the following expression for α_k:

$$\alpha_k = \frac{\boldsymbol{p}^{(k)^T} \boldsymbol{r}^{(k)}}{\boldsymbol{p}^{(k)^T} A \boldsymbol{p}^{(k)}} \tag{11.49}$$

Conjugate Gradient Method

Given $\boldsymbol{x}^{(0)}$, compute $\boldsymbol{r}^{(0)} = \boldsymbol{b} - A\boldsymbol{x}^{(0)}, \boldsymbol{p}^{(0)} = \boldsymbol{r}^{(0)}$

While(Stopping criterion is not satisfied)

> Compute step lenght $\alpha_k = \frac{\boldsymbol{p}^{(k)^T} \boldsymbol{r}^{(k)}}{\boldsymbol{p}^{(k)^T} A \boldsymbol{p}^{(k)}}$
>
> Update the solution: $\boldsymbol{x}^{(k+1)} = \boldsymbol{x}^{(k)} + \alpha_k \boldsymbol{p}^{(k)}$
>
> Update the residual: $\boldsymbol{r}^{(k+1)} = \boldsymbol{r}^{(k)} - \alpha_k A \boldsymbol{p}^{(k)}$
>
> Compute the conjugate coefficient $\beta_k = \frac{\big(A\boldsymbol{p}^{(k)}\big)^T \boldsymbol{r}^{(k+1)}}{\big(A\boldsymbol{p}^{(k)}\big)^T \boldsymbol{p}^{(k)}}$
>
> Update the direction: $\boldsymbol{p}^{(k+1)} = \boldsymbol{r}^{(k+1)} - \beta_k \boldsymbol{p}^{(k)}$

11.7.1 The Preconditioned Conjugate Gradient Method

If P is a symmetric and positive definite matrix, the preconditioned conjugate gradient method(PCG) consists of applying the CG method to the preconditioned system:

$$P^{-\frac{1}{2}} A P^{-\frac{1}{2}} \boldsymbol{y} = P^{-\frac{1}{2}} \boldsymbol{b}, \text{with } \boldsymbol{y} = P^{-\frac{1}{2}} \boldsymbol{x} \tag{11.50}$$

Preconditioned Conjugate Gradient Method

Given $\boldsymbol{x}^{(0)}$, compute $\boldsymbol{r}^{(0)} = \boldsymbol{b} - A\boldsymbol{x}^{(0)}, \boldsymbol{z}^{(0)} = P^{-1}\boldsymbol{r}^{(0)}, \boldsymbol{p}^{(0)} = \boldsymbol{z}^{(0)}$

While(Stopping criterion is not satisfied)

> Compute step length $\alpha_k = \frac{\boldsymbol{p}^{(k)^T} \boldsymbol{r}^{(k)}}{\boldsymbol{p}^{(k)^T} A \boldsymbol{p}^{(k)}}$
>
> Update the solution: $\boldsymbol{x}^{(k+1)} = \boldsymbol{x}^{(k)} + \alpha_k \boldsymbol{p}^{(k)}$
>
> Update the residual: $\boldsymbol{r}^{(k+1)} = \boldsymbol{r}^{(k)} - \alpha_k A \boldsymbol{p}^{(k)}$
>
> Solve the preconditioned system: $P\boldsymbol{z}^{(k+1)} = \boldsymbol{r}^{(k+1)}$
>
> Compute the conjugate coefficient $\beta_k = \frac{\boldsymbol{z}^{(k+1)^T} \boldsymbol{r}^{(k+1)}}{\boldsymbol{z}^{(k+1)^T} \boldsymbol{z}^{(k)}}$
>
> Update the direction: $\boldsymbol{p}^{(k+1)} = \boldsymbol{z}^{(k+1)} + \beta_k \boldsymbol{p}^{(k)}$

Thm Finite-Termination Property **theorem 11.7.1**

For an $n \times n$ SPD matrix A , the CG algorithm converges to the exact solution $x = A^{-1}b$ in at most n iterations in exact arithmetic.

Thm Error Bounds **theorem 11.7.2**

Let A be an $n \times n$ SPD matrix and let $x^{(k)}$ be the iterate of the CG method at the k-th step. Then, the error $e^{(k)} = x - x^{(k)}$ satisfies the following error bounds:

$$\|e^{(k)}\|_A \leq \frac{2c^k}{1 + c^{2k}} \|e^{(0)}\|_A \tag{11.51}$$

where $c = \frac{\sqrt{K(A)}-1}{\sqrt{K(A)}+1}$ and $K(A)$ is the condition number of A.

11.8 Methods Based on Krylov Subspace Iterations

Def Krylov Subspace **definition 11.8.1**

Given a nonsingular matrix $A \in \mathbb{R}(n \times n)$ and $\boldsymbol{y} \in \mathbb{R}^n$, $\boldsymbol{y} \neq \mathbf{0}$, the mth Krylov subspace $K_m(A, \boldsymbol{y})$ generated by A and $\boldsymbol{y}$ is

$$K_m(A, \boldsymbol{y}) = \text{span}\left\{\boldsymbol{y}, A\boldsymbol{y}, A^2\boldsymbol{y}, ..., A^{m-1}\boldsymbol{y}\right\} \tag{11.52}$$

Consider the Richardson method Eq. (11.31) with $P = I$; the residual at the k-th step can be related to the initial residual as

$$\boldsymbol{r}^{(k)} = \prod_{j=0}^{k-1} (I - \alpha_j A)\boldsymbol{r}^{(0)} \tag{11.53}$$

So start at $\boldsymbol{r}^{(k)} = p_k A\boldsymbol{r}^{(0)}$, where $p_k(A)$ is a polynomial in A of degree k.

In an analogous manner as for Eq. (11.53), it is seen that the iterate $\boldsymbol{x}^{(k)}$ of the Richardson method is given by

$$\boldsymbol{x}^{(k)} = \boldsymbol{x}^{(0)} + \sum_{j=0}^{k-1} \alpha_j \boldsymbol{r}^{(j)} \tag{11.54}$$

so that $\boldsymbol{x}^{(k)}$ belongs to the following space

$$W_k = \left\{\boldsymbol{v} = \boldsymbol{x}^{(0)} + \boldsymbol{y}, \boldsymbol{y} \in K_k\left(A, \boldsymbol{r}^{(0)}\right)\right\} \tag{11.55}$$

In the nonpreconditioned Richardson method we are thus looking for an approximate solution to $\boldsymbol{x}$ in the space W_k. More generally, we can think of devising methods that search for approximate solutions of the form

$$\boldsymbol{x}^{(k)} = \boldsymbol{x}^{(0)} + q_{k-1}(A)\boldsymbol{r}^{(0)} \tag{11.56}$$

where q_{k-1} is polynomial selected in such a way that $\boldsymbol{x}^{(k)}$ be, in a sense that must be made precise, the best approximation of $\boldsymbol{x}$ in W_k. A method that looks for a solution of the form Eq. (11.53) is called a *Krylov subspace method.*

12 Solving large scale eigenvalue problems

12.1 Eigenvalues and Eigenvectors

Given a square matrix $A \in \mathbb{C}^{n\times n}$, find a scalar λ and a nonzero vector $x \in \mathbb{C}^n$ such that:

$$Ax = \lambda x \tag{12.1}$$

where:

1. The vector x is the *eigenvector*, And the scalar λ is the *eigenvalue*
2. The set of all the eigenvalues of a matrix A is called the *spectrum* of A, denoted by $\sigma(A)$.
3. The maximum modulus of all the eigenvalues is called the *spectral radius* of A and is denoted by $\rho(A) = \max_{\{\lambda\in\sigma(A)\}}|\lambda|$.

Remarks

1. The eigenvalues of a matrix are the roots of the characteristic polynomial $\det(A - \lambda I) = 0$.
2. From the Fundamental Theorem of Algebra, an $n \times n$ matrix has exactly n eigenvalues, counting multiplicities.
3. Each λ_i may be real but in general is a complex number
4. The eigenvalues $\lambda_1, \lambda_2, ..., \lambda_n$may not all have distinct values
5. Rayleigh quotient: $\lambda_i = \frac{x_i^H A x_i}{x_i^H x_i}$

12.2 The Power Method

Let $A \in \mathbb{C}^{n\times n}$ be a diagonalizable matrix and let $X \in \mathbb{C}^{n\times n}$ be the matrix of its eigenvectors $\boldsymbol{x_i}$. for $i = 1, ..., n$. Let us also suppose that the eigenvalues of A are ordered as

$$|\lambda_1| > |\lambda_2| \geq ... \geq |\lambda_n| \tag{12.2}$$

Where λ_1 has algebraic multiplicity equal to 1. Under these assumptions, λ_1, is called the *dominant eigenvalue* of A.

Given an arbitrary initial vector $\boldsymbol{q}_0 \in \mathbb{C}^n$ with unitary Euclidean norm, consider for $k = 1, 2, ...$ the following iteration based on the computation of powers of matrices, commonly known as the *power method*:

$$\begin{aligned} \boldsymbol{z}^{(k)} &= A\boldsymbol{q}^{(k-1)} \\ \boldsymbol{q}^{(k)} &= \frac{\boldsymbol{z}^{(k)}}{\|\boldsymbol{z}^{(\boldsymbol{k})}\|} \\ \nu^{(k)} &= \boldsymbol{q}^{((k))^H} A\boldsymbol{q}^{(k)} \end{aligned} \tag{12.3}$$

THE POWER METHOD

$\boldsymbol{q}_0$ =some initial vector with $\|\boldsymbol{q_0}\| = 1$
For $k = 1, 2, ...$
| Apply A: $\boldsymbol{z}^{(k)} = A\boldsymbol{q}^{(k-1)}$

THE POWER METHOD

Normalize: $\boldsymbol{q}^{(k)} = \frac{\boldsymbol{z}^{(k)}}{\|\boldsymbol{z}^{(k)}\|}$
Compute Rayleigh quotient: $\nu^{(k)} = \boldsymbol{q}^{((k))^H} A\boldsymbol{q}^{(k)}$

Let us analyze the convergence of the power method. By induction on k, we have that:

$$\boldsymbol{q}^{(k)} = A^k \frac{\boldsymbol{q}^{(0)}}{\|\boldsymbol{A^k q^{(0)}}\|}, k \geq 1 \tag{12.4}$$

This relation explains the role played by the powers of A in the method. Because A is diagonalizable, its eigenvectors $\boldsymbol{x}_i$ form a basis of $\mathbb{C}^n$ and we can write:

$$\boldsymbol{q}^{(0)} = \sum_{i=1}^{n} \alpha_i \boldsymbol{x}_i \tag{12.5}$$

Moreover, since $A\boldsymbol{x}_i = \lambda_i \boldsymbol{x}_i$, we have:

$$\begin{aligned} A^k \boldsymbol{q}^{(0)} &= \sum_{i=1}^{n} \alpha_i \lambda_i^k \boldsymbol{x}_i \\ &= \alpha_1 \lambda_1^k \left(\boldsymbol{x}_1 + \sum_{i=2}^{n} \frac{\alpha_i}{\alpha_1} \left(\frac{\lambda_i}{\lambda_1} \right)^k \boldsymbol{x}_i \right) \end{aligned} \tag{12.6}$$

Since $\left|\frac{\lambda_i}{\lambda_1}\right| < 1$ for $i = 2, ..., n$, as k increases the term $\sum_{i=2}^{n} \frac{\alpha_i}{\alpha_1} \left(\frac{\lambda_i}{\lambda_1}\right)^k \boldsymbol{x}_i$ tends to assume an increasingly significant component in the direction of the eigenvector $\boldsymbol{x}_1$, while ts components in the other directions $\boldsymbol{x}_j$ decrease.

As $k \to \infty$, the vector $\boldsymbol{q}^{(k)}$ thus aligns itself along the direction of eigenvector $\boldsymbol{x}_1$, and the following error estimate holds at each step k.

Thm Convergence of the Power Method **theorem 12.2.1**

Let $A \in \mathbb{C}^{n \times n}$ be a diagonalizable matrix whose dominant eigenvalue is λ_1. Assuming that $\alpha_1 \neq 0$, there exists a constant $C > 0$ such that:

$$\|\tilde{\boldsymbol{q}}^{(\boldsymbol{k})} - \boldsymbol{x_1}\| \leq C \left(|\frac{\lambda_2}{\lambda_1}| \right)^k, k \geq 1 \tag{12.7}$$

where:

$$\tilde{\boldsymbol{q}}^{(k)} = \boldsymbol{x}_1 + \sum_{i=2}^{n} \frac{\alpha_i}{\alpha_1} \left(\frac{\lambda_i}{\lambda_1} \right)^k \boldsymbol{x}_i, k = 1, 2, ... \tag{12.8}$$

Estimate Eq. (12.7) expresses the convergence of the sequence of $\tilde{\boldsymbol{q}}^{(k)}$ towards the eigenvector $\boldsymbol{x}_1$ of A. Therefore the sequence of Rayleigh quotients

$$\tilde{\boldsymbol{q}}^{((k))^H} A \tilde{\boldsymbol{q}}^{(k)} / \|\tilde{\boldsymbol{q}}^{(\boldsymbol{k})}\| = \left(\boldsymbol{q}^{(k)}\right)^H A \boldsymbol{q}^{(k)} = \nu^{(k)} \tag{12.9}$$

will converge to the dominant eigenvalue λ_1 of A. As a consequence, and the convergence will be faster when the ratio $|\frac{\lambda_2}{\lambda_1}|$ is smaller.

12.3 Deflation

Let A be an $n \times n$ real symmetric matrix with isolated non-zero eigenvalues

$$|\lambda_1| > |\lambda_2| > ... > |\lambda_n| > 0 \tag{12.10}$$

and eigenvector $\boldsymbol{v}_1, ..., \boldsymbol{v}_n$. The goal of **deflation** is to build a modified matrix that has only $\lambda_2, ..., \lambda_n$ as eigenvalues. Suppose we have obtained λ_1 and $\boldsymbol{v}_1$. This goal is achieved by defining the 'deflated' matrix B as

$$B = A - \lambda_1 \boldsymbol{v}_1 \boldsymbol{x} \tag{12.11}$$

where $\boldsymbol{x}$ is any vector such that

$$\boldsymbol{v}_1^T \boldsymbol{x} = 1 \tag{12.12}$$

12.4 The Inverse Power Method

We look for an approximation of the eigenvalue of a matrix $A \in \mathbb{C}^{n \times n}$ which is *closest* to a given number $\mu \in \mathbb{C}$, where $\mu \notin \sigma(A)$. For this, the power iteration is applied to the matrix $\left(M_\mu\right)^{-1} = (A - \mu I)^{-1}$, yielding the so-called *inverse iteration* or *inverse power method*. The number μ is called the *shift* of the method.

The eigenvalues of M_u^{-1} are $\xi = (\lambda_i - \mu)^{-1}$, let us assume that there exists an integer m such that

$$|\lambda_m - \mu| < |\lambda_i - \mu| \tag{12.13}$$

Given an arbitrary initial vector $\boldsymbol{q}_0 \in \mathbb{C}^n$ with unitary Euclidean norm, for $k = 1, 2, ...$ the following sequence is constructed:

$$\begin{aligned} (A - \mu I)\boldsymbol{z}^{(k)} &= \boldsymbol{q}^{(k-1)} \\ \boldsymbol{q}^{(k)} &= \frac{\boldsymbol{z}^{(k)}}{\|\boldsymbol{z}^{(k)}\|} \\ \nu^{(k)} &= \boldsymbol{q}^{((k))^H} A \boldsymbol{q}^{(k)} \end{aligned} \tag{12.14}$$

THE INVERSE POWER METHOD

$\boldsymbol{q}_0$ =some initial vector with $\|\boldsymbol{q_0}\| = 1$
For $k = 1, 2, ...$
  Solve linear equation $(A - \mu I)$: $\boldsymbol{z}^{(k)} = (A - \mu I)\boldsymbol{q}^{(k-1)}$
  Normalize: $\boldsymbol{q}^{(k)} = \frac{\boldsymbol{z}^{(k)}}{\|\boldsymbol{z}^{(k)}\|}$
  Compute Rayleigh quotient: $\nu^{(k)} = \boldsymbol{q}^{((k))^H} A \boldsymbol{q}^{(k)}$

Notice that the eigenvectors of M_μ are the same as those of A since $M_\mu = X(\Lambda - \mu I_n)X^{-1}$, where $\Lambda = \text{diag}(\lambda_1, ..., \lambda_n)$. For this reason, the Rayleigh quotient in Eq. (12.14) is computed directly on the matrix A. The main difference with respect to Eq. (12.3) is that at each step k a linear system with coefficient matrix $M_\mu = A - \mu I$ *must be solved.*

12.5 QR Iterative Method

In this section we present some iterative techniques for simultaneously approximating all the eigenvalues of a given matrix A. The basic idea consists of reducing A, by means of suitable similarity transformations, into a form for which the calculation of the eigenvalues is easier than on the starting matrix.

Let $A \in \mathbb{C}^{n\times n}$. The QR algorithm computes an upper triangular matrix T and a unitary matrix U such that $A = UTU^H$.

QR Iterative Method

Set $A^{(0)} = A$, $U^{(0)} = I$

While(Stopping criterion is not satisfied)

Compute the QR factorization: $A^{(k-1)} = Q^{(k)}R^{(k)}$

Update the matrix: $A^{(k)} = R_k Q_k$

Update the unitary matrix: $U^{(k)} = U^{(k-1)}Q^{(k)}$

Return $T = A^{(k)}, U = U^{(k)}$

Notice that

1. $A^{(k)} = R^{(k)}Q^{(k)} = \left[Q^{(k)}\right]^H A^{(k-1)}Q^{(k)}$, therefore $A^{(k)}$ is similar to $A^{(k-1)}$.
2. Moreover, from the above observaSon, we have $A^{(k)} = \left[Q^{(k)}\right]^H ... \left[Q^{(1)}\right]^H A^{(0)} \left[Q^{(1)}\right] ... Q^{(k)}$.

12.6 The Lanzos algorithm

The Lanczos algorithm can be used to efficiently find the extremal eigenvalues(maximum and minimum) of a symmetric matrix A of size $n \times n$.

It is based on computing the following decomposition of A:

$$A = Q^T T Q \tag{12.15}$$

where Q is an orthonormal basis of vectors $\boldsymbol{q}_1, ..., \boldsymbol{q}_n$ and T is a tridiagonal matrix of size $n \times n$. The decomposiSon always exists and is unique provided that $\boldsymbol{q}_1$ has been specified.

We know that $T = Q^T AQ$ which gives

$$\begin{aligned} \alpha_k &= \boldsymbol{q}_k^T A \boldsymbol{q}_k \\ \beta_k &= \boldsymbol{q}_{k+1}^T A \boldsymbol{q}_k \end{aligned} \tag{12.16}$$

The full decompostion is obtained by imposing $AQ = QT$:

$$\begin{aligned} A\boldsymbol{q}_1 &= \alpha_1 \boldsymbol{q}_1 + \beta_1 \boldsymbol{q}_2 \\ A\boldsymbol{q}_2 &= \beta_1 \boldsymbol{q}_1 + \alpha_2 \boldsymbol{q}_2 + \beta_2 \boldsymbol{q}_3 \\ &... \\ A\boldsymbol{q}_n &= \beta_{n-1} \boldsymbol{q}_{n-1} + \alpha_n \boldsymbol{q}_n \end{aligned} \tag{12.17}$$

13 Numerical methods for overdetermined linear systems of equations

Overdetermined linear systems of equations are systems of equations in which the number of equations is greater than the number of unknowns. In this case, the system is said to be *overdetermined*. When the problems are linear there is a very clean and simple way to find the optimum, if we adopt the sum-of-squares error metric.

13.1 QR Factorization for rectangular matrices

Let $A \in \mathbb{R}(m \times n)$ be a rectangular matrix, then

$$A = QR \tag{13.1}$$

where $Q \in \mathbb{R}(m \times m)$ is an orthogonal matrix and $R \in \mathbb{R}(m \times n)$ is an upper trapezoidal matrix.

One version of the QR factorization is *reduced QR factorization*. Let A be an $m \times n$ matrix. The reduced QR factorization of A is a factorization of the form:

$$A = \hat{Q}\hat{R} \tag{13.2}$$

where $Q \in \mathbb{R}(m \times n)$ is an rectangular matrix and $R \in \mathbb{R}(n \times n)$ is an upper triangular matrix.

13.2 Linear Regression

If there were no experimental uncertainty the model would fit the data exactly, but since there is noise the best we can do is minimise the error. The problem is:

$$\min_{\alpha_0,\alpha_1} \sum_{i=1}^{m} \boldsymbol{e}_i^2 = \min_{\alpha_0,\alpha_1} \sum_{i=1}^{m} (\alpha_0 + \alpha_1 T_i - L_i)^2 \tag{13.3}$$

The above problem is equivalent to the following:

$$\min_{\alpha_0,\alpha_1} \|\boldsymbol{e}\|^2 = \min_{\alpha_0,\alpha_1} \|\boldsymbol{A\alpha} - (\boldsymbol{b})\|^2 \tag{13.4}$$

with $A = [1, T_1; 1, T_2; ...; 1, T_m]$ and $\boldsymbol{\alpha} = [\alpha_0, \alpha_1]^T$, $\boldsymbol{b} = [L_1, L_2, ..., L_m]^T$.

13.3 The Least Squares Solution

The mathematical problem reads: given a matrix $A \in \mathbb{R}^{m \times n}$ and a vector $\boldsymbol{b} \in \mathbb{R}^m$, find the vector $\boldsymbol{x} \in \mathbb{R}^n$ such that:

$$A\boldsymbol{x} = \boldsymbol{b} \tag{13.5}$$

We notice that generally the above problems has no solution (in the classical sense) unless the right side b is an element of range(A). We need a new concept of solution, the basic approach is to look for an $\boldsymbol{x}$ that makes $A\boldsymbol{x}$ close to $\boldsymbol{b}$.

Def Least Squares Solution **definition 13.3.1**

Given a matrix $A \in \mathbb{R}^{m\times n}, m \geq n$, we say that $\boldsymbol{x}^* \in \mathbb{R}^n$ is a solution of the linear system $A\boldsymbol{x} = \boldsymbol{b}$ in the sense of *least squares* if

$$\Phi(\boldsymbol{x}^*) = \min_{\{\boldsymbol{y}\in\mathbb{R}^n\}} \Phi(\boldsymbol{y}) \tag{13.6}$$

where $\Phi(\boldsymbol{y}) = \|A\boldsymbol{y} - \boldsymbol{b}\|^2$. The problem thus consists of minimising the Euclidean norm of the residual.

The solution $\boldsymbol{x}^*$ can be found by imposing the condition that the gradient of function Φ must be zero at $\boldsymbol{x}^*$.

From the definition we have

$$\nabla\Phi(\boldsymbol{x}^*) = \nabla\|A\boldsymbol{x}^* - \boldsymbol{b}\|^2 = 2A^T(A\boldsymbol{x}^* - \boldsymbol{b}) = 0 \tag{13.7}$$

from which it follows that $A^TA\boldsymbol{x}^* = A^T\boldsymbol{b}$.

Thm Exists and Unique **theorem 13.3.1**

The system of normal equations is nonsingular if A has full rank and, in such a case, the least squares solution exists and is unique.

We notice that $B = A^TA$ is a SPD matrix. Thus, in order to solve the normal equations, one could first compute the Cholesky factorization of B, by solving two triangular systems:

$$\begin{aligned} R\boldsymbol{z} &= A^T\boldsymbol{b} \\ R\boldsymbol{x}^* &= \boldsymbol{z} \end{aligned} \tag{13.8}$$

However, A^TA is very badly conditioned and, due to roundoff errors, the computation of A^TA may be affected by a loss of significant digits, with a consequent loss of positive definiteness or nonsingularity of the matrix!

Thm Solution in Reduced QR Factorization **theorem 13.3.2**

Let $A \in \mathbb{R}^{m\times n}, m \geq n$, **be a full rank matrix**, and let $A = \hat{Q}\hat{R}$ be the reduced QR factorization of A. Then the unique solution in the least-square sense of the system $A\boldsymbol{x} = \boldsymbol{b}$ is given by

$$\boldsymbol{x}^* = \hat{R}^{-1}\hat{Q}^T\boldsymbol{b} \tag{13.9}$$

Moreover, the minimum of the function Φ is given by

$$\Phi(\boldsymbol{x}^*) = \sum_{i=n+1}^{m} \left[(Q^T\boldsymbol{b})_i\right]^2 \tag{13.10}$$

13.4 SVD

If A does not have full rank, the above solution techniques above fail. In this case if $\boldsymbol{x}^*$ is a solution in the least square sense, the vector $\boldsymbol{x}^* + \boldsymbol{z}$, with $\boldsymbol{z} \in N(A)$, is also a solution. We must therefore

introduce a further constraint to enforce the uniqueness of the solution. Typically, one requires that $\boldsymbol{x}^*$ has minimal Euclidean norm, so that the least squares problem can be formulated as:

$$\text{Find } \boldsymbol{x}^* \text{ such that } \|\boldsymbol{A}\boldsymbol{x}^* - \boldsymbol{b}\|^2 \leq \min_{\boldsymbol{x}\in\mathbb{R}^n} \|\boldsymbol{A}\boldsymbol{x} - \boldsymbol{b}\|^2 \tag{13.11}$$

This problem is consistent with our formulation. If A has full rank, since in this case the solution in the least-square sense exists and is unique it necessarily must have minimal Euclidean norm. The tool for solving Eq. (13.11) is the singular value decomposition(SVD).

Def pseudo-inverse **definition 13.4.1**

Suppose that $A \in \mathbb{R}^{m\times n}$ has rank equal to r and let $A = U\Sigma V^T$ be the SVD of A. The *pseudo-inverse* of A is the matrix

$$A^\dagger = V\Sigma^\dagger U^T \tag{13.12}$$

where $\Sigma^\dagger$ is the $n \times m$ matrix obtained by taking the reciprocal of the nonzero singular values of Σ and transposing the result.

The matrix $A^\dagger$ is also called the *generalized inverse* of A. And if $n = m = \text{rank}(A)$, then $A^\dagger = A^{-1}$.

13.5 Ways to compute $\hat{x}$

13.5.1 Gram-Schmidt with Column Pivoting

13.5.2 Householder Transformation

The good way to create an exactly orthogonal Q is to build it that way, as Householder did:

Def Householder Transformation **definition 13.5.1**

Given a vector $\boldsymbol{x} \in \mathbb{R}^n, \boldsymbol{x} \neq 0$, the *Householder transformation* H is the matrix

$$H = I - 2\frac{\boldsymbol{x}\boldsymbol{x}^T}{\boldsymbol{x}^T\boldsymbol{x}} \tag{13.13}$$

Properties:

- H is orthogonal: $H^T H = I$
- H is symmetric: $H^T = H$

Let $\boldsymbol{y} \in \mathbb{R}^n$. Then the transformation $H\boldsymbol{y} = \boldsymbol{y} - 2\frac{\boldsymbol{x}^T y}{\boldsymbol{x}^T\boldsymbol{x}}$ reflects $\boldsymbol{y}$ about the hyperplane orthogonal to $\boldsymbol{x}$. Let's make some special choices for y:

- If $\boldsymbol{y} = \boldsymbol{x}$, then $H\boldsymbol{x} = \boldsymbol{x} - 2\frac{\boldsymbol{x}^T x}{\boldsymbol{x}^T\boldsymbol{x}} = \boldsymbol{x} - 2\boldsymbol{x} = \boldsymbol{x}$
- If $\boldsymbol{y}$ is orthogonal to $\boldsymbol{x}$, then $H\boldsymbol{y} = \boldsymbol{y}$

We want to use Householder transformations to obtain $A = QR$, the idea is to transform the matrix column by column

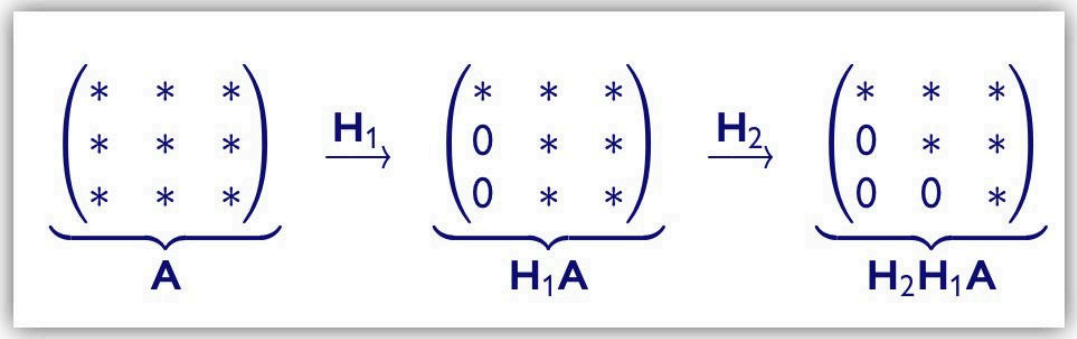


Figure 13.7: Householder QR factorization

- Define $R := H_2 H_1 A$, then R is upper triangular and $A = H_1^T H_2^2 R$.
- Define $Q := H_1^T H_2^T$, then Q is orthogonal and $A = QR$.

Let $A \in \mathbb{R}^{n \times n}$, we choose H_1 such that:

$$\boldsymbol{u} = \boldsymbol{a}_1 = \begin{bmatrix} a_1 \\ a_2 \\ \dots \\ a_n \end{bmatrix} \overset{H_1}{\longrightarrow} \boldsymbol{v} = \begin{bmatrix} \alpha \\ 0 \\ \dots \\ 0 \end{bmatrix} = \alpha \boldsymbol{e}_1 \tag{13.14}$$

$\boldsymbol{u}$ and $\boldsymbol{v}$ must have same length, so

$$\|\boldsymbol{u}\|_2 = \|\boldsymbol{v}\|_2 = \|\boldsymbol{a_1}\|_2 = |\alpha| \tag{13.15}$$

Conclusion:

$$\alpha = \pm \|\boldsymbol{a_1}\|_2, \boldsymbol{v} = \pm \|\boldsymbol{a_1}\|_2 \boldsymbol{e}_1 \tag{13.16}$$

We need to take $H_1 = I - 2\frac{\boldsymbol{x}\boldsymbol{x}^T}{\boldsymbol{x}^T\boldsymbol{x}}$, with

$$\boldsymbol{x} = \boldsymbol{u} - \boldsymbol{v} = \boldsymbol{a_1} \pm \|\boldsymbol{a_1}\|_2 \boldsymbol{e}_1 \tag{13.17}$$

Let us choose the sign such that no cancellation occurs in computing

$$\boldsymbol{x} := \begin{cases} \boldsymbol{a_1} - \|\boldsymbol{a_1}\|_2 \boldsymbol{e}_1 \text{ if } a_{11} > 0 \\ \\ \boldsymbol{a_1} + \|\boldsymbol{a_1}\|_2 \boldsymbol{e}_1 \text{ if } a_{11} < 0 \end{cases} \tag{13.18}$$

After the first Householder transformation:

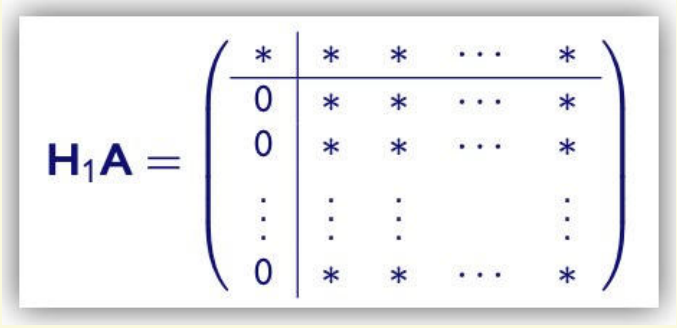


Figure 13.8: Householder QR factorization, first step

Let B be the $(n-1) \times (n-1)$ matrix that you get by deleting the first row and column of A. We can write $B = (\boldsymbol{b}_1 | \boldsymbol{b}_2 | ... | \boldsymbol{b}_n)$, where $\boldsymbol{b}_i$ is the i-th column of B. Transform the first column $\boldsymbol{b}_1$ in $\beta \boldsymbol{e}_1$ using the Householder transformation

$$\widehat{H}_2 = I - 2\frac{\widehat{\boldsymbol{x}}_2 \widehat{\boldsymbol{x}}_2^T}{\widehat{\boldsymbol{x}}_2^T \widehat{\boldsymbol{x}}_2} \tag{13.19}$$

Define H_2 as

$$H_2 = \begin{bmatrix} 1 & 0 \\ 0 & \widehat{H}_2 \end{bmatrix} \in \mathbb{R}^{n \times n} \tag{13.20}$$

After the second transformation:

$$\mathbf{H_2 H_1 A} = \left(\begin{array}{cc|ccc} * & * & * & \cdots & * \\ 0 & * & * & \cdots & * \\ \hline 0 & 0 & * & \cdots & * \\ \vdots & \vdots & \vdots & & \vdots \\ 0 & 0 & * & \cdots & * \end{array}\right)$$

Figure 13.9: Householder QR factorization, second step

In general, we need $n-1$ Householder transformations to get $H_{n-1}...H_2 H_1 A = R$, and hence $A = QR$.

13.5.3 Givens Rotation